Heidelberg Educational Numerics Library

Version 0.27 (from 15 March 2021)

Generated by Doxygen 1.9.1

1 Hierarchical Index 1
1.1 Class Hierarchy . . . 1

2 Class Index 3
2.1 Class List . . . 3

3 File Index 5
3.1 File List . . . 5

4 Class Documentation 7
4.1 hdnum::Banach Class Reference . . . 7
4.1.1 Detailed Description . . . 7
4.2 hdnum::SparseMatrix< REAL >::builder Class Reference . . . 8
4.3 hdnum::SparseMatrix< REAL >::column_index_iterator Class Reference . . . 8
4.4 hdnum::SparseMatrix< REAL >::const_column_index_iterator Class Reference . . . 9
4.5 hdnum::SparseMatrix< REAL >::const_row_iterator Class Reference . . . 9
4.6 hdnum::oc::OpCounter< F >::Counters Struct Reference . . . 10
4.6.1 Detailed Description . . . 11
4.7 hdnum::DenseMatrix< REAL > Class Template Reference . . . 11
4.7.1 Detailed Description . . . 14
4.7.2 Member Function Documentation . . . 14
4.7.2.1 colsize() . . . 14
4.7.2.2 mm() . . . 14
4.7.2.3 mv() . . . 15
4.7.2.4 operator()() . . . 16
4.7.2.5 operator∗() [1/2] . . . 17
4.7.2.6 operator∗() [2/2] . . . 17
4.7.2.7 operator∗=() . . . 18
4.7.2.8 operator+() . . . 19
4.7.2.9 operator+=() . . . 19
4.7.2.10 operator-() . . . 20
4.7.2.11 operator-=() . . . 20
4.7.2.12 operator/=() . . . 21
4.7.2.13 operator=() [1/2] . . . 21
4.7.2.14 operator=() [2/2] . . . 22
4.7.2.15 rowsize() . . . 22
4.7.2.16 sc() . . . 22
4.7.2.17 scientific() . . . 23
4.7.2.18 sr() . . . 23
4.7.2.19 sub() . . . 24
4.7.2.20 transpose() . . . 24
4.7.2.21 umm() . . . 25
4.7.2.22 umv() [1/2] . . . 26

4.7.2.23 umv() [2/2] . . . 26
4.7.2.24 update() . . . 27
4.7.3 Friends And Related Function Documentation . . . 28
4.7.3.1 identity() . . . 28
4.7.3.2 readMatrixFromFileDat() . . . 28
4.7.3.3 readMatrixFromFileMatrixMarket() . . . 29
4.7.3.4 spd() . . . 30
4.7.3.5 vandermonde() . . . 30
4.8 hdnum::DIRK< M, S > Class Template Reference . . . 31
4.8.1 Detailed Description . . . 32
4.8.2 Constructor & Destructor Documentation . . . 32
4.8.2.1 DIRK() [1/2] . . . 32
4.8.2.2 DIRK() [2/2] . . . 33
4.9 hdnum::EE< M > Class Template Reference . . . 33
4.9.1 Detailed Description . . . 34
4.10 hdnum::ErrorException Class Reference . . . 34
4.10.1 Detailed Description . . . 34
4.11 hdnum::Exception Class Reference . . . 35
4.11.1 Detailed Description . . . 35
4.12 hdnum::GenericNonlinearProblem< Lambda, Vec > Class Template Reference . . . 35
4.12.1 Detailed Description . . . 36
4.13 hdnum::Heun2< M > Class Template Reference . . . 36
4.13.1 Detailed Description . . . 37
4.14 hdnum::Heun3< M > Class Template Reference . . . 37
4.14.1 Detailed Description . . . 38
4.15 hdnum::IE< M, S > Class Template Reference . . . 38
4.15.1 Detailed Description . . . 39
4.16 hdnum::ImplicitRungeKuttaStepProblem< M > Class Template Reference . . . 39
4.16.1 Detailed Description . . . 40
4.17 hdnum::InvalidStateException Class Reference . . . 40
4.17.1 Detailed Description . . . 41
4.18 hdnum::IOError Class Reference . . . 41
4.18.1 Detailed Description . . . 41
4.19 hdnum::Kutta3< M > Class Template Reference . . . 41
4.19.1 Detailed Description . . . 42
4.20 hdnum::MathError Class Reference . . . 42
4.20.1 Detailed Description . . . 43
4.21 hdnum::ModifiedEuler< M > Class Template Reference . . . 43
4.21.1 Detailed Description . . . 44
4.22 hdnum::Newton Class Reference . . . 44
4.22.1 Detailed Description . . . 45
4.23 hdnum::NotImplemented Class Reference . . . 45

4.23.1 Detailed Description . . . 46
4.24 hdnum::oc::OpCounter< F > Class Template Reference . . . 46
4.24.1 Detailed Description . . . 47
4.25 hdnum::OutOfMemoryError Class Reference . . . 47
4.25.1 Detailed Description . . . 48
4.26 hdnum::RangeError Class Reference . . . 48
4.26.1 Detailed Description . . . 48
4.27 hdnum::RE< M, S > Class Template Reference . . . 48
4.27.1 Detailed Description . . . 49
4.28 hdnum::RKF45< M > Class Template Reference . . . 50
4.28.1 Detailed Description . . . 50
4.29 hdnum::SparseMatrix< REAL >::row_iterator Class Reference . . . 51
4.30 hdnum::RungeKutta< M, S > Class Template Reference . . . 51
4.30.1 Detailed Description . . . 52
4.31 hdnum::RungeKutta4< M > Class Template Reference . . . 53
4.31.1 Detailed Description . . . 53
4.32 hdnum::SGrid< N, DF, dimension > Class Template Reference . . . 54
4.32.1 Detailed Description . . . 55
4.32.2 Constructor & Destructor Documentation . . . 55
4.32.2.1 SGrid() . . . 55
4.32.3 Member Function Documentation . . . 55
4.32.3.1 getNeighborIndex() . . . 56
4.33 hdnum::SparseMatrix< REAL > Class Template Reference . . . 56
4.33.1 Detailed Description . . . 58
4.33.2 Constructor & Destructor Documentation . . . 58
4.33.2.1 SparseMatrix() . . . 58
4.33.3 Member Function Documentation . . . 59
4.33.3.1 begin() [1/2] . . . 59
4.33.3.2 begin() [2/2] . . . 59
4.33.3.3 cbegin() . . . 59
4.33.3.4 cend() . . . 59
4.33.3.5 colsize() . . . 60
4.33.3.6 end() [1/2] . . . 60
4.33.3.7 end() [2/2] . . . 60
4.33.3.8 identity() . . . 61
4.33.3.9 matchingIdentity() . . . 61
4.33.3.10 mv() . . . 61
4.33.3.11 norm_infty() . . . 62
4.33.3.12 operator∗() . . . 62
4.33.3.13 operator∗=() . . . 62
4.33.3.14 operator/=() . . . 63
4.33.3.15 rowsize() . . . 63

4.33.3.16 scientific() . . . 63
4.33.3.17 umv() . . . 64
4.33.4 Friends And Related Function Documentation . . . 64
4.33.4.1 identity() . . . 64
4.34 hdnum::SquareRootProblem< N > Class Template Reference . . . 65
4.34.1 Detailed Description . . . 65
4.35 hdnum::StationarySolver< M > Class Template Reference . . . 66
4.35.1 Detailed Description . . . 66
4.36 hdnum::SystemError Class Reference . . . 66
4.36.1 Detailed Description . . . 67
4.37 hdnum::Timer Class Reference . . . 67
4.37.1 Detailed Description . . . 67
4.38 hdnum::TimerError Class Reference . . . 68
4.38.1 Detailed Description . . . 68
4.39 hdnum::Vector< REAL > Class Template Reference . . . 68
4.39.1 Detailed Description . . . 70
4.39.2 Member Function Documentation . . . 70
4.39.2.1 operator∗() . . . 70
4.39.2.2 operator+() . . . 71
4.39.2.3 operator-() . . . 71
4.39.2.4 operator=() . . . 72
4.39.2.5 scientific() . . . 72
4.39.2.6 sub() . . . 73
4.39.2.7 two_norm() . . . 73
4.39.3 Friends And Related Function Documentation . . . 74
4.39.3.1 fill() . . . 74
4.39.3.2 gnuplot() . . . 74
4.39.3.3 operator<<() . . . 75
4.39.3.4 readVectorFromFile() . . . 75
4.39.3.5 unitvector() . . . 76

5 File Documentation **77**
5.1 src/exceptions.hh File Reference . . . 77
5.1.1 Detailed Description . . . 78
5.1.2 Macro Definition Documentation . . . 78
5.1.2.1 HDNUM_ERROR . . . 78
5.1.2.2 HDNUM_THROW . . . 78
5.2 src/lr.hh File Reference . . . 79
5.2.1 Detailed Description . . . 79
5.3 src/newton.hh File Reference . . . 80
5.3.1 Detailed Description . . . 80
5.3.2 Function Documentation . . . 80

5.3.2.1 getNonlinearProblem() . . . 80
5.4 src/ode.hh File Reference . . . 81
5.4.1 Detailed Description . . . 82
5.5 src/opcounter.hh File Reference . . . 82
5.5.1 Detailed Description . . . 85
5.6 src/pde.hh File Reference . . . 85
5.6.1 Detailed Description . . . 85
5.7 src/precision.hh File Reference . . . 85
5.7.1 Detailed Description . . . 86
5.8 src/qr.hh File Reference . . . 86
5.8.1 Detailed Description . . . 86
5.8.2 Function Documentation . . . 86
5.8.2.1 gram_schmidt() . . . 87
5.8.2.2 modified_gram_schmidt() . . . 87
5.8.2.3 permute_forward() . . . 88
5.8.2.4 qr_gram_schmidt() . . . 88
5.8.2.5 qr_gram_schmidt_pivoting() . . . 89
5.8.2.6 qr_gram_schmidt_simple() . . . 90
5.9 src/qrhousholder.hh File Reference . . . 91
5.9.1 Detailed Description . . . 92
5.9.2 Function Documentation . . . 92
5.9.2.1 qrhousholder() . . . 92
5.9.2.2 qrhousholderexplizitQ() . . . 92
5.10 src/rungekutta.hh File Reference . . . 93
5.10.1 Detailed Description . . . 93
5.10.2 Function Documentation . . . 93
5.10.2.1 ordertest() . . . 93
5.11 src/timer.hh File Reference . . . 94
5.11.1 Detailed Description . . . 94

Index **95**

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

hdnum::Banach . . . 7
hdnum::SparseMatrix< REAL >::builder . . . 8
hdnum::SparseMatrix< REAL >::column_index_iterator . . . 8
hdnum::SparseMatrix< REAL >::const_column_index_iterator . . . 9
hdnum::SparseMatrix< REAL >::const_row_iterator . . . 9
hdnum::oc::OpCounter< F >::Counters . . . 10
hdnum::DenseMatrix< REAL > . . . 11
hdnum::DenseMatrix< number_type > . . . 11
hdnum::DIRK< M, S > . . . 31
hdnum::EE< M > . . . 33
hdnum::Exception . . . 35
- hdnum::ErrorException . . . 34
- hdnum::IOError . . . 41
- hdnum::InvalidStateException . . . 40
- hdnum::MathError . . . 42
- hdnum::NotImplemented . . . 45
- hdnum::RangeError . . . 48
- hdnum::SystemError . . . 66
  - hdnum::OutOfMemoryError . . . 47
  - hdnum::TimerError . . . 68

hdnum::GenericNonlinearProblem< Lambda, Vec > . . . 35
hdnum::Heun2< M > . . . 36
hdnum::Heun3< M > . . . 37
hdnum::IE< M, S > . . . 38
hdnum::ImplicitRungeKuttaStepProblem< M > . . . 39
hdnum::Kutta3< M > . . . 41
hdnum::ModifiedEuler< M > . . . 43
hdnum::Newton . . . 44
hdnum::oc::OpCounter< F > . . . 46
hdnum::RE< M, S > . . . 48
hdnum::RKF45< M > . . . 50
hdnum::SparseMatrix< REAL >::row_iterator . . . 51
hdnum::RungeKutta< M, S > . . . 51
hdnum::RungeKutta4< M > . . . 53
hdnum::SGrid< N, DF, dimension > . . . 54

hdnum::SparseMatrix< REAL > 56
hdnum::SquareRootProblem< N > 65
hdnum::StationarySolver< M > 66
hdnum::Timer 67
std::vector
hdnum::Vector< hdnum::Vector< number_type > > 68
hdnum::Vector< number_type > 68
hdnum::Vector< size_type > 68
hdnum::Vector< REAL > 68

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

hdnum::Banach
Solve nonlinear problem using a fixed point iteration . . . 7
hdnum::SparseMatrix< REAL >::builder . . . 8
hdnum::SparseMatrix< REAL >::column_index_iterator . . . 8
hdnum::SparseMatrix< REAL >::const_column_index_iterator . . . 9
hdnum::SparseMatrix< REAL >::const_row_iterator . . . 9
hdnum::oc::OpCounter< F >::Counters
Struct storing the number of operations . . . 10
hdnum::DenseMatrix< REAL >
Class with mathematical matrix operations . . . 11
hdnum::DIRK< M, S >
Implementation of a general Diagonal Implicit Runge-Kutta method . . . 31
hdnum::EE< M >
Explicit Euler method as an example for an ODE solver . . . 33
hdnum::ErrorException
General Error . . . 34
hdnum::Exception
Base class for Exceptions . . . 35
hdnum::GenericNonlinearProblem< Lambda, Vec >
A generic problem class that can be set up with a lambda defining F(x)=0 . . . 35
hdnum::Heun2< M >
Heun method (order 2 with 2 stages) . . . 36
hdnum::Heun3< M >
Heun method (order 3 with 3 stages) . . . 37
hdnum::IE< M, S >
Implicit Euler using Newton's method to solve nonlinear system . . . 38
hdnum::ImplicitRungeKuttaStepProblem< M >
Nonlinear problem we need to solve to do one step of an implicit Runge Kutta method . . . 39
hdnum::InvalidStateException
Default exception if a function was called while the object is not in a valid state for that function 40
hdnum::IOError
Default exception class for I/O errors . . . 41
hdnum::Kutta3< M >
Kutta method (order 3 with 3 stages) . . . 41
hdnum::MathError
Default exception class for mathematical errors . . . 42

hdnum::ModifiedEuler< M >
Modified Euler method (order 2 with 2 stages) 43
hdnum::Newton
Solve nonlinear problem using a damped Newton method 44
hdnum::NotImplemented
Default exception for dummy implementations 45
hdnum::oc::OpCounter< F > 46
hdnum::OutOfMemoryError
Default exception if memory allocation fails 47
hdnum::RangeError
Default exception class for range errors 48
hdnum::RE< M, S >
Adaptive one-step method using Richardson extrapolation 48
hdnum::RKF45< M >
Adaptive Runge-Kutta-Fehlberg method 50
hdnum::SparseMatrix< REAL >::row_iterator 51
hdnum::RungeKutta< M, S >
Classical Runge-Kutta method (order n with n stages) 51
hdnum::RungeKutta4< M >
Classical Runge-Kutta method (order 4 with 4 stages) 53
hdnum::SGrid< N, DF, dimension >
Structured Grid for Finite Differences 54
hdnum::SparseMatrix< REAL >
Sparse matrix Class with mathematical matrix operations 56
hdnum::SquareRootProblem< N >
Example class for a nonlinear model F(x) = 0; 65
hdnum::StationarySolver< M >
Stationary problem solver. E.g. for elliptic problmes 66
hdnum::SystemError
Default exception class for OS errors 66
hdnum::Timer
A simple stop watch 67
hdnum::TimerError
Exception thrown by the Timer class 68
hdnum::Vector< REAL >
Class with mathematical vector operations 68

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

src/**densematrix.hh** . . . . . ??
src/exceptions.hh
A few common exception classes 77
src/lr.hh
This file implements LU decomposition 79
src/newton.hh
Newton's method with line search 80
src/ode.hh
Solvers for ordinary differential equations 81
src/opcounter.hh
This file implements an operator counting class 82
src/pde.hh
Solvers for partial differential equations 85
src/precision.hh
Find machine precision for given float type 85
src/qr.hh
This file implements QR decomposition using Gram-Schmidt method 86
src/qrhousholder.hh
This file implements QR decomposition using housholder transformation 91
src/rungekutta.hh 93
src/**sgrid.hh** . . . . . ??
src/**sparsematrix.hh** . . . . . ??
src/timer.hh
A simple timing class 94
src/**vector.hh** . . . . . ??

Chapter 4

Class Documentation

4.1 hdnum::Banach Class Reference

Solve nonlinear problem using a fixed point iteration.

`#include <newton.hh>`

Public Member Functions

- Banach ()

  constructor stores reference to the model
- void set_maxit (size_type n)

  maximum number of iterations before giving up
- void set_sigma (double sigma_)

  damping parameter
- void set_linesearchsteps (size_type n)

  maximum number of steps in linesearch before giving up
- void set_verbosity (size_type n)

  control output given 0=nothing, 1=summary, 2=every step, 3=include line search
- void set_abslimit (double l)

  basolute limit for defect
- void set_reduction (double l)

  reduction factor
- template<class M >
  void solve (const M &model, Vector< typename M::number_type > &x) const

  do one step
- bool **has_converged** () const

4.1.1 Detailed Description

Solve nonlinear problem using a fixed point iteration.

solve F(x) = 0.

$$x = x - \sigma * F(x)$$

The documentation for this class was generated from the following file:

- src/newton.hh

4.2 hdnum::SparseMatrix< REAL >::builder Class Reference

Public Member Functions

- **builder** (size_type new_m_rows, size_type new_m_cols)
- **builder** (const std::initializer_list< std::initializer_list< REAL >> &v)
- std::pair< typename std::map< size_type, REAL >::iterator, bool > **addEntry** (size_type i, size_type j, REAL value)
- std::pair< typename std::map< size_type, REAL >::iterator, bool > **addEntry** (size_type i, size_type j)
- bool **operator==** (const SparseMatrix::builder &other) const
- bool **operator!=** (const SparseMatrix::builder &other) const
- size_type **colsize** () const noexcept
- size_type **rowsize** () const noexcept
- size_type **setNumCols** (size_type new_m_cols) noexcept
- size_type **setNumRows** (size_type new_m_rows)
- void **clear** () noexcept
- std::string **to_string** () const
- SparseMatrix **build** ()

The documentation for this class was generated from the following file:

- src/sparsematrix.hh

4.3 hdnum::SparseMatrix< REAL >::column_index_iterator Class Reference

Public Types

- using **self_type** = column_index_iterator
- using **difference_type** = std::ptrdiff_t
- using **value_type** = std::pair< REAL &, size_type const & >
- using **pointer** = value_type *
- using **reference** = value_type &
- using **iterator_category** = std::bidirectional_iterator_tag

Public Member Functions

- **column_index_iterator** (typename std::vector< REAL >::iterator valIter, std::vector< size_type >::iterator colIndicesIter)
- self_type & **operator++** ()
- self_type & **operator++** (int junk)
- value_type **operator*** ()
- value_type::first_type **value** ()
- value_type::second_type **index** ()
- bool **operator==** (const self_type &other)
- bool **operator!=** (const self_type &other)

The documentation for this class was generated from the following file:

- src/sparsematrix.hh

4.4 hdnum::SparseMatrix< REAL >::const_column_index_iterator Class Reference

Public Types

- using **self_type** = const_column_index_iterator
- using **difference_type** = std::ptrdiff_t
- using **value_type** = std::pair< REAL const &, size_type const & >
- using **pointer** = value_type *
- using **reference** = value_type &
- using **iterator_category** = std::bidirectional_iterator_tag

Public Member Functions

- **const_column_index_iterator** (typename std::vector< REAL >::const_iterator valIter, std::vector< size_type >::const_iterator colIndicesIter)
- self_type & **operator++** ()
- self_type **operator++** (int junk)
- value_type **operator*** ()
- value_type::first_type **value** ()
- value_type::second_type **index** ()
- bool **operator==** (const self_type &other)
- bool **operator!=** (const self_type &other)

The documentation for this class was generated from the following file:

- src/sparsematrix.hh

4.5 hdnum::SparseMatrix< REAL >::const_row_iterator Class Reference

Public Types

- using **self_type** = const_row_iterator
- using **difference_type** = std::ptrdiff_t
- using **value_type** = self_type
- using **pointer** = self_type *
- using **reference** = self_type &
- using **iterator_category** = std::bidirectional_iterator_tag

Public Member Functions

- **const_row_iterator** (std::vector< size_type >::const_iterator rowPtrIter, std::vector< size_type >::const↩_iterator colIndicesIter, typename std::vector< REAL >::const_iterator valIter)
- const_column_iterator **begin** () const
- const_column_iterator **end** () const
- const_column_index_iterator **ibegin** () const
- const_column_index_iterator **iend** () const
- const_column_iterator **cbegin** () const
- const_column_iterator **cend** () const
- self_type & **operator++** ()
- self_type & **operator++** (int junk)
- self_type & **operator+=** (difference_type offset)
- self_type & **operator-=** (difference_type offset)
- self_type **operator-** (difference_type offset)
- self_type **operator+** (difference_type offset)
- reference **operator[]** (difference_type offset)
- bool **operator**< (const self_type &other)
- bool **operator**> (const self_type &other)
- self_type & **operator**∗ ()
- bool **operator==** (const self_type &rhs)
- bool **operator!=** (const self_type &rhs)

Friends

- self_type **operator+** (const difference_type &offset, const self_type &sec)

The documentation for this class was generated from the following file:

- src/sparsematrix.hh

4.6 hdnum::oc::OpCounter< F >::Counters Struct Reference

Struct storing the number of operations.

```
#include <opcounter.hh>
```

Public Member Functions

- void **reset** ()
- template<typename Stream >
  void reportOperations (Stream &os, bool doReset=false)

  Report operations to stream object.
- size_type totalOperationCount (bool doReset=false)

  Get total number of operations.
- Counters & **operator+=** (const Counters &rhs)
- Counters **operator-** (const Counters &rhs)

Public Attributes

- size_type **addition_count**
- size_type **multiplication_count**
- size_type **division_count**
- size_type **exp_count**
- size_type **pow_count**
- size_type **sin_count**
- size_type **sqrt_count**
- size_type **comparison_count**

4.6.1 Detailed Description

template<typename F>
struct hdnum::oc::OpCounter< F >::Counters

Struct storing the number of operations.

The documentation for this struct was generated from the following file:

- src/opcounter.hh

4.7 hdnum::DenseMatrix< REAL > Class Template Reference

Class with mathematical matrix operations.

```
#include <densematrix.hh>
```

Public Types

- typedef std::size_t size_type

  Type used for array indices.
- typedef std::vector< REAL > **VType**
- typedef VType::const_iterator **ConstVectorIterator**
- typedef VType::iterator **VectorIterator**

Public Member Functions

- DenseMatrix ()

  default constructor (empty Matrix)
- DenseMatrix (const std::size_t _rows, const std::size_t _cols, const REAL def_val=0)

  constructor
- DenseMatrix (const std::initializer_list< std::initializer_list< REAL >> &v)

  constructor from initializer list
- DenseMatrix (const hdnum::SparseMatrix< REAL > &other)

  constructor from hdnum::SparseMatrix
- void **addNewRow** (const hdnum::Vector< REAL > &rowvector)
- size_t rowsize () const

  get number of rows of the matrix
- size_t colsize () const

  get number of columns of the matrix
- bool **scientific** () const
- void scientific (bool b) const

  Switch between floating point (default=true) and fixed point (false) display.
- std::size_t iwidth () const

  get index field width for pretty-printing
- std::size_t width () const

  get data field width for pretty-printing
- std::size_t precision () const

  get data precision for pretty-printing
- void iwidth (std::size_t i) const

  set index field width for pretty-printing
- void width (std::size_t i) const

  set data field width for pretty-printing
- void precision (std::size_t i) const

  set data precision for pretty-printing
- REAL & operator() (const std::size_t row, const std::size_t col)

  (i,j)-operator for accessing entries of a (m x n)-matrix directly
- const REAL & operator() (const std::size_t row, const std::size_t col) const

  read-access on matrix element A_ij using A(i,j)
- const ConstVectorIterator operator[ ] (const std::size_t row) const

  read-access on matrix element A_ij using A[i][j]
- VectorIterator operator[ ] (const std::size_t row)

  write-access on matrix element A_ij using A[i][j]
- DenseMatrix & operator= (const DenseMatrix &A)

  assignment operator
- DenseMatrix & operator= (const REAL value)

  assignment from a scalar value
- DenseMatrix sub (size_type i, size_type j, size_type rows, size_type cols)

  Submatrix extraction.
- DenseMatrix transpose () const

  Transposition.
- DenseMatrix & operator+= (const DenseMatrix &B)

  Addition assignment.
- DenseMatrix & operator-= (const DenseMatrix &B)

  Subtraction assignment.
- DenseMatrix & operator∗= (const REAL s)

Scalar multiplication assignment.

- DenseMatrix & operator/= (const REAL s)

  Scalar division assignment.
- void update (const REAL s, const DenseMatrix &B)

  Scaled update of a Matrix.
- template<class V >
  void mv (Vector< V > &y, const Vector< V > &x) const

  matrix vector product y = A∗x
- template<class V >
  void umv (Vector< V > &y, const Vector< V > &x) const

  update matrix vector product y += A∗x
- template<class V >
  void umv (Vector< V > &y, const V &s, const Vector< V > &x) const

  update matrix vector product y += sA∗x
- void mm (const DenseMatrix< REAL > &A, const DenseMatrix< REAL > &B)

  assign to matrix product C = A∗B to matrix C
- void umm (const DenseMatrix< REAL > &A, const DenseMatrix< REAL > &B)

  add matrix product A∗B to matrix C
- void sc (const Vector< REAL > &x, std::size_t k)

  set column: make x the k'th column of A
- void sr (const Vector< REAL > &x, std::size_t k)

  set row: make x the k'th row of A
- REAL norm_infty () const

  compute row sum norm
- REAL norm_1 () const

  compute column sum norm
- Vector< REAL > operator∗ (const Vector< REAL > &x) const

  vector = matrix ∗ vector
- DenseMatrix operator∗ (const DenseMatrix &x) const

  matrix = matrix ∗ matrix
- DenseMatrix operator+ (const DenseMatrix &x) const

  matrix = matrix + matrix
- DenseMatrix operator- (const DenseMatrix &x) const

  matrix = matrix - matrix

Related Functions

(Note that these are not member functions.)

- template<class T >
  void identity (DenseMatrix< T > &A)
- template<typename REAL >
  void spd (DenseMatrix< REAL > &A)
- template<typename REAL >
  void vandermonde (DenseMatrix< REAL > &A, const Vector< REAL > x)
- template<typename REAL >
  void readMatrixFromFileDat (const std::string &filename, DenseMatrix< REAL > &A)

  Read matrix from a text file.
- template<typename REAL >
  void readMatrixFromFileMatrixMarket (const std::string &filename, DenseMatrix< REAL > &A)

  Read matrix from a matrix market file.

4.7.1 Detailed Description

template<typename REAL>
class hdnum::DenseMatrix< REAL >

Class with mathematical matrix operations.

4.7.2 Member Function Documentation

4.7.2.1 colsize()

```
template<typename REAL >
size_t hdnum::DenseMatrix< REAL >::colsize ( ) const  [inline]
```

get number of columns of the matrix

Example:

```
hdnum::DenseMatrix<double> A(4,5);
size_t nColumns = A.colsize();
std::cout « "Matrix A has " « nColumns « " columns." « std::endl;
```

Output:

```
Matrix A has 5 columns.
```

4.7.2.2 mm()

```
template<typename REAL >
void hdnum::DenseMatrix< REAL >::mm (
            const DenseMatrix< REAL > & A,
            const DenseMatrix< REAL > & B )  [inline]
```

assign to matrix product C = A∗B to matrix C

Implements C = A∗B where A and B are matrices

Parameters

in	*A*	constant reference to a DenseMatrix
in	*B*	constant reference to a DenseMatrix

Example:

```
hdnum::DenseMatrix<double> A(2,6,1.0);
hdnum::DenseMatrix<double> B(6,3,-1.0);
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(6);          // use at least 6 columns for displaying
matrix entries A.precision(3);      // display 3 digits behind the point
```

```
std::cout « "A =" « A « std::endl;
std::cout « "B =" « B « std::endl;
hdnum::DenseMatrix<double> C(2,3);
C.mm(A,B);
std::cout « "C = A*B =" « C « std::endl;
```

Output:

```
A =
0       1       2       3       4       5
0   1.000   1.000   1.000   1.000   1.000   1.000
1   1.000   1.000   1.000   1.000   1.000   1.000

B =
0       1       2
0  -1.000 -1.000 -1.000
1  -1.000 -1.000 -1.000
2  -1.000 -1.000 -1.000
3  -1.000 -1.000 -1.000
4  -1.000 -1.000 -1.000
5  -1.000 -1.000 -1.000

C = A*B =
0       1       2
0  -6.000 -6.000 -6.000
1  -6.000 -6.000 -6.000
```

4.7.2.3 mv()

```
template<typename REAL >
template<class V >
void hdnum::DenseMatrix< REAL >::mv (
            Vector< V > & y,
            const Vector< V > & x ) const  [inline]
```

matrix vector product y = A∗x

Implements y = A∗x where x and y are a vectors and A is a matrix

Parameters

in	*y*	reference to the resulting Vector
in	*x*	constant reference to a Vector

Example:

```
hdnum::Vector<double> x(3,10.0);
hdnum::Vector<double> y(2);
hdnum::DenseMatrix<double> A(2,3,1.0);
x.scientific(false); // fixed point representation for all Vector objects
A.scientific(false); // fixed point representation for all DenseMatrix
objects
std::cout « "A =" « A « std::endl;
std::cout « "x =" « x « std::endl;
A.mv(y,x);
std::cout « "y = A*x =" « y « std::endl;
```

Output:

```
A =
0           1           2
```

```
0       1.000      1.000      1.000
1       1.000      1.000      1.000

x =
[ 0]     10.0000000
[ 1]     10.0000000
[ 2]     10.0000000

y = A*x =
[ 0]     30.0000000
[ 1]     30.0000000
```

4.7.2.4 operator()()

```
template<typename REAL >
REAL& hdnum::DenseMatrix< REAL >::operator() (
            const std::size_t row,
            const std::size_t col )  [inline]
```

(i,j)-operator for accessing entries of a (m x n)-matrix directly

Parameters

in	*row*	row index (0...m-1)
in	*col*	column index (0...n-1)

Example:

```
hdnum::DenseMatrix<double> A(4,4);
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(8); A.precision(3);
identity(A);  // Defines the identity matrix of the same dimension
std::cout « "A=" « A « std::endl;
std::cout « "reading A(0,0)=" « A(0,0) « std::endl;
std::cout « "resetting A(0,0) and A(2,3)..." « std::endl;
A(0,0) = 1.234;
A(2,3) = 432.1;
std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0        1        2        3
0     1.000    0.000    0.000    0.000
1     0.000    1.000    0.000    0.000
2     0.000    0.000    1.000    0.000
3     0.000    0.000    0.000    1.000

reading A(0,0)=1.000
resetting A(0,0) and A(2,3)...
A=
0        1        2        3
0     1.234    0.000    0.000    0.000
1     0.000    1.000    0.000    0.000
2     0.000    0.000    1.000  432.100
3     0.000    0.000    0.000    1.000
```

4.7.2.5 operator∗() [1/2]

```
template<typename REAL >
DenseMatrix hdnum::DenseMatrix< REAL >::operator* (
            const DenseMatrix< REAL > & x ) const  [inline]
```

matrix = matrix ∗ matrix

Parameters

in	*x*	constant reference to a DenseMatrix

Example:

```
hdnum::DenseMatrix<double> A(3,3,2.0);
hdnum::DenseMatrix<double> B(3,3,4.0);
hdnum::DenseMatrix<double> C(3,3);
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(8); A.precision(1);
std::cout « "A=" « A « std::endl;
std::cout « "B=" « B « std::endl;
C=A*B;
std::cout « "C=A*B=" « C « std::endl;
```

Output:

```
A=
0         1         2
0       2.0       2.0       2.0
1       2.0       2.0       2.0
2       2.0       2.0       2.0

B=
0         1         2
0       4.0       4.0       4.0
1       4.0       4.0       4.0
2       4.0       4.0       4.0

C=A*B=
0         1         2
0      24.0      24.0      24.0
1      24.0      24.0      24.0
2      24.0      24.0      24.0
```

4.7.2.6 operator∗() [2/2]

```
template<typename REAL >
Vector<REAL> hdnum::DenseMatrix< REAL >::operator* (
            const Vector< REAL > & x ) const  [inline]
```

vector = matrix ∗ vector

Parameters

in	*x*	constant reference to a Vector

Example:

```
hdnum::Vector<double> x(3,4.0);
```

```
hdnum::DenseMatrix<double> A(3,3,2.0);
hdnum::Vector<double> y(3);
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(8); A.precision(1);
x.scientific(false); // fixed point representation for all Vector objects
x.width(8);
x.precision(1);
std::cout « "A=" « A « std::endl;
std::cout « "x=" « x « std::endl;
y=A*x;
std::cout « "y=A*x" « y « std::endl;
```

Output:

```
A=
0         1         2
0        2.0       2.0       2.0
1        2.0       2.0       2.0
2        2.0       2.0       2.0

x=
[ 0]      4.0
[ 1]      4.0
[ 2]      4.0

y=A*x
[ 0]     24.0
[ 1]     24.0
[ 2]     24.0
```

4.7.2.7 operator*=()

```
template<typename REAL >
DenseMatrix& hdnum::DenseMatrix< REAL >::operator*= (
            const REAL s )  [inline]
```

Scalar multiplication assignment.

Implements A *= s where s is a scalar

Parameters

in	*s*	scalar value to multiply with

Example:

```
double s = 0.5;
hdnum::DenseMatrix<double> A(2,3,1.0);
std::cout « "A=" « A « std::endl;
A *= s;
std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0          1          2
0   1.000e+00  1.000e+00  1.000e+00
1   1.000e+00  1.000e+00  1.000e+00

0.5*A =
0          1          2
0   5.000e-01  5.000e-01  5.000e-01
1   5.000e-01  5.000e-01  5.000e-01
```

4.7.2.8 operator+()

```
template<typename REAL >
DenseMatrix hdnum::DenseMatrix< REAL >::operator+ (
            const DenseMatrix< REAL > & x ) const  [inline]
```

matrix = matrix + matrix

Parameters

in	*x*	constant reference to a DenseMatrix

Example:

```
hdnum::DenseMatrix<double> A(3,3,2.0);
hdnum::DenseMatrix<double> B(3,3,4.0);
hdnum::DenseMatrix<double> C(3,3);
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(8); A.precision(1);
std::cout « "A=" « A « std::endl;
std::cout « "B=" « B « std::endl;
C=A+B;
std::cout « "C=A+B=" « C « std::endl;
```

Output:

```
A=
0         1         2
0        2.0       2.0       2.0
1        2.0       2.0       2.0
2        2.0       2.0       2.0

B=
0         1         2
0        4.0       4.0       4.0
1        4.0       4.0       4.0
2        4.0       4.0       4.0

C=A+B=
0         1         2
0        6.0       6.0       6.0
1        6.0       6.0       6.0
2        6.0       6.0       6.0
```

4.7.2.9 operator+=()

```
template<typename REAL >
DenseMatrix& hdnum::DenseMatrix< REAL >::operator+= (
            const DenseMatrix< REAL > & B )  [inline]
```

Addition assignment.

Implements A += B matrix addition

Parameters

in	*B*	another Matrix

4.7.2.10 operator-()

```
template<typename REAL >
DenseMatrix hdnum::DenseMatrix< REAL >::operator- (
            const DenseMatrix< REAL > & x ) const  [inline]
```

matrix = matrix - matrix

Parameters

in	*x*	constant reference to a DenseMatrix

Example:

```
hdnum::DenseMatrix<double> A(3,3,2.0);
hdnum::DenseMatrix<double> B(3,3,4.0);
hdnum::DenseMatrix<double> C(3,3);
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(8); A.precision(1);
std::cout « "A=" « A « std::endl;
std::cout « "B=" « B « std::endl;
C=A-B;
std::cout « "C=A-B=" « C « std::endl;
```

Output:

```
A=
0         1         2
0       2.0       2.0       2.0
1       2.0       2.0       2.0
2       2.0       2.0       2.0

B=
0         1         2
0       4.0       4.0       4.0
1       4.0       4.0       4.0
2       4.0       4.0       4.0

C=A-B=
0         1         2
0      -2.0      -2.0      -2.0
1      -2.0      -2.0      -2.0
2      -2.0      -2.0      -2.0
```

4.7.2.11 operator-=()

```
template<typename REAL >
DenseMatrix& hdnum::DenseMatrix< REAL >::operator-= (
            const DenseMatrix< REAL > & B )  [inline]
```

Subtraction assignment.

Implements A -= B matrix subtraction

Parameters

in	*B*	another matrix

4.7.2.12 operator/=()

```
template<typename REAL >
DenseMatrix& hdnum::DenseMatrix< REAL >::operator/= (
            const REAL s )  [inline]
```

Scalar division assignment.

Implements A /= s where s is a scalar

Parameters

in	*s*	scalar value to multiply with

Example:

```
double s = 0.5;
hdnum::DenseMatrix<double> A(2,3,1.0);
std::cout « "A=" « A « std::endl;
A /= s;
std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0          1          2
0   1.000e+00  1.000e+00  1.000e+00
1   1.000e+00  1.000e+00  1.000e+00

A/0.5 =
0          1          2
0   2.000e+00  2.000e+00  2.000e+00
1   2.000e+00  2.000e+00  2.000e+00
```

4.7.2.13 operator=() [1/2]

```
template<typename REAL >
DenseMatrix& hdnum::DenseMatrix< REAL >::operator= (
            const DenseMatrix< REAL > & A )  [inline]
```

assignment operator

Example:

```
hdnum::DenseMatrix<double> A(4,4);
spd(A);
hdnum::DenseMatrix<double> B(4,4);
B = A;
std::cout « "B=" « B « std::endl;
```

Output:

```
B=
0          1          2          3
0   4.000e+00 -1.000e+00 -2.500e-01 -1.111e-01
1  -1.000e+00  4.000e+00 -1.000e+00 -2.500e-01
2  -2.500e-01 -1.000e+00  4.000e+00 -1.000e+00
3  -1.111e-01 -2.500e-01 -1.000e+00  4.000e+00
```

4.7.2.14 operator=() [2/2]

```
template<typename REAL >
DenseMatrix& hdnum::DenseMatrix< REAL >::operator= (
            const REAL value )  [inline]
```

assignment from a scalar value

Example:

```
hdnum::DenseMatrix<double> A(2,3);
A = 5.432;
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(8); A.precision(3); std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0        1        2
0     5.432    5.432    5.432
1     5.432    5.432    5.432
```

4.7.2.15 rowsize()

```
template<typename REAL >
size_t hdnum::DenseMatrix< REAL >::rowsize ( ) const  [inline]
```

get number of rows of the matrix

Example:

```
hdnum::DenseMatrix<double> A(4,5);
size_t nRows = A.rowsize();
std::cout « "Matrix A has " « nRows « " rows." « std::endl;
```

Output:

```
Matrix A has 4 rows.
```

4.7.2.16 sc()

```
template<typename REAL >
void hdnum::DenseMatrix< REAL >::sc (
            const Vector< REAL > & x,
            std::size_t k )  [inline]
```

set column: make x the k'th column of A

Parameters

in	x	constant reference to a Vector
in	k	number of the column of A to be set

Example:

```
hdnum::Vector<double> x(2,434.0);
hdnum::DenseMatrix<double> A(2,6);
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(8); A.precision(1);
std::cout « "original A=" « A « std::endl;
A.sc(x,3);   // redefine fourth column of the matrix
std::cout « "modified A=" « A « std::endl;
```

Output:

```
original A=
0        1        2        3        4        5
0      0.0      0.0      0.0      0.0      0.0      0.0
1      0.0      0.0      0.0      0.0      0.0      0.0

modified A=
0        1        2        3        4        5
0      0.0      0.0      0.0    434.0      0.0      0.0
1      0.0      0.0      0.0    434.0      0.0      0.0
```

4.7.2.17 scientific()

```
template<typename REAL >
void hdnum::DenseMatrix< REAL >::scientific (
            bool b ) const  [inline]
```

Switch between floating point (default=true) and fixed point (false) display.

Example:

```
hdnum::DenseMatrix<double> A(4,4);
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(8); A.precision(3); identity(A);  // Defines the identity
matrix of the same dimension std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0        1        2        3
0    1.000    0.000    0.000    0.000
1    0.000    1.000    0.000    0.000
2    0.000    0.000    1.000    0.000
3    0.000    0.000    0.000    1.000
```

4.7.2.18 sr()

```
template<typename REAL >
void hdnum::DenseMatrix< REAL >::sr (
            const Vector< REAL > & x,
            std::size_t k )  [inline]
```

set row: make x the k'th row of A

Parameters

in	*x*	constant reference to a Vector
in	*k*	number of the row of A to be set

Example:

```
hdnum::Vector<double> x(3,434.0);
hdnum::DenseMatrix<double> A(3,3);
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(8); A.precision(1);
std::cout « "original A=" « A « std::endl;
A.sr(x,1);   // redefine second row of the matrix
std::cout « "modified A=" « A « std::endl;
```

Output:

```
original A=
0        1        2
0       0.0      0.0      0.0
1       0.0      0.0      0.0
2       0.0      0.0      0.0

modified A=
0        1        2
0       0.0      0.0      0.0
1     434.0    434.0    434.0
2       0.0      0.0      0.0
```

4.7.2.19 sub()

```
template<typename REAL >
DenseMatrix hdnum::DenseMatrix< REAL >::sub (
            size_type i,
            size_type j,
            size_type rows,
            size_type cols )  [inline]
```

Submatrix extraction.

Returns a new matrix that is a subset of the components of the given matrix.

Parameters

in	*i*	first row index of the new matrix
in	*j*	first column index of the new matrix
in	*rows*	row size of the new matrix, i.e. it has components [i,i+rows-1]
in	*cols*	column size of the new matrix, i.e. it has components [j,j+cols-1]

4.7.2.20 transpose()

```
template<typename REAL >
DenseMatrix hdnum::DenseMatrix< REAL >::transpose ( ) const  [inline]
```

Transposition.

Return the transposed as a new matrix.

4.7.2.21 umm()

```
template<typename REAL >
void hdnum::DenseMatrix< REAL >::umm (
            const DenseMatrix< REAL > & A,
            const DenseMatrix< REAL > & B )  [inline]
```

add matrix product A∗B to matrix C

Implements C += A∗B where A, B and C are matrices

Parameters

in	*A*	constant reference to a DenseMatrix
in	*B*	constant reference to a DenseMatrix

Example:

```
hdnum::DenseMatrix<double> A(2,6,1.0);
hdnum::DenseMatrix<double> B(6,3,-1.0);
hdnum::DenseMatrix<double> C(2,3,0.5);
A.scientific(false); // fixed point representation for all DenseMatrix
objects A.width(6); A.precision(3);
std::cout « "C =" « C « std::endl;
std::cout « "A =" « A « std::endl;
std::cout « "B =" « B « std::endl;
C.umm(A,B);
std::cout « "C + A*B =" « C « std::endl;
```

Output:

```
C =
0       1       2
0   0.500   0.500   0.500
1   0.500   0.500   0.500

A =
0       1       2       3       4       5
0   1.000   1.000   1.000   1.000   1.000   1.000
1   1.000   1.000   1.000   1.000   1.000   1.000

B =
0       1       2
0  -1.000  -1.000  -1.000
1  -1.000  -1.000  -1.000
2  -1.000  -1.000  -1.000
3  -1.000  -1.000  -1.000
4  -1.000  -1.000  -1.000
5  -1.000  -1.000  -1.000

C + A*B =
0       1       2
0  -5.500  -5.500  -5.500
1  -5.500  -5.500  -5.500
```

4.7.2.22 umv() [1/2]

```
template<typename REAL >
template<class V >
void hdnum::DenseMatrix< REAL >::umv (
            Vector< V > & y,
            const V & s,
            const Vector< V > & x ) const  [inline]
```

update matrix vector product y += sA∗x

Implements y += sA∗x where s is a scalar value, x and y are a vectors and A is a matrix

Parameters

in	*y*	reference to the resulting Vector
in	*s*	constant reference to a number type
in	*x*	constant reference to a Vector

Example:

```
double s=0.5;
hdnum::Vector<double> x(3,10.0);
hdnum::Vector<double> y(2,5.0);
hdnum::DenseMatrix<double> A(2,3,1.0);
x.scientific(false); // fixed point representation for all Vector objects
A.scientific(false); // fixed point representation for all DenseMatrix
objects
std::cout « "y =" « y « std::endl;
std::cout « "A =" « A « std::endl;
std::cout « "x =" « x « std::endl;
A.umv(y,s,x);
std::cout « "y = s*A*x =" « y « std::endl;
```

Output:

```
y =
[ 0]      5.0000000
[ 1]      5.0000000

A =
0          1          2
0       1.000      1.000      1.000
1       1.000      1.000      1.000

x =
[ 0]     10.0000000
[ 1]     10.0000000
[ 2]     10.0000000

y = s*A*x =
[ 0]     20.0000000
[ 1]     20.0000000
```

4.7.2.23 umv() [2/2]

```
template<typename REAL >
template<class V >
void hdnum::DenseMatrix< REAL >::umv (
            Vector< V > & y,
            const Vector< V > & x ) const  [inline]
```

update matrix vector product y += A∗x

Implements y += A∗x where x and y are a vectors and A is a matrix

Parameters

in	*y*	reference to the resulting Vector
in	*x*	constant reference to a Vector

Example:

```
hdnum::Vector<double> x(3,10.0);
hdnum::Vector<double> y(2,5.0);
hdnum::DenseMatrix<double> A(2,3,1.0);
x.scientific(false); // fixed point representation for all Vector objects
A.scientific(false); // fixed point representation for all DenseMatrix
objects
std::cout « "y =" « y « std::endl;
std::cout « "A =" « A « std::endl;
std::cout « "x =" « x « std::endl;
A.umv(y,x);
std::cout « "y = A*x =" « y « std::endl;
```

Output:

```
y =
[ 0]      5.0000000
[ 1]      5.0000000

A =
0          1          2
0       1.000      1.000      1.000
1       1.000      1.000      1.000

x =
[ 0]     10.0000000
[ 1]     10.0000000
[ 2]     10.0000000

y + A*x =
[ 0]     35.0000000
[ 1]     35.0000000
```

4.7.2.24 update()

```
template<typename REAL >
void hdnum::DenseMatrix< REAL >::update (
            const REAL s,
            const DenseMatrix< REAL > & B )  [inline]
```

Scaled update of a Matrix.

Implements A += s∗B where s is a scalar and B a matrix

Parameters

in	*s*	scalar value to multiply with
in	*B*	another matrix

Example:

```
double s = 0.5;
hdnum::DenseMatrix<double> A(2,3,1.0);
hdnum::DenseMatrix<double> B(2,3,2.0);
```

```
A.update(s,B);
std::cout « "A + s*B =" « A « std::endl;
```

Output:

```
A + s*B =
0          1          2
0       1.500      1.500      1.500
1       1.500      1.500      1.500
```

4.7.3 Friends And Related Function Documentation

4.7.3.1 identity()

```
template<class T >
void identity (
            DenseMatrix< T > & A )  [related]
```

Function: make identity matrix

```
template<class T>
inline void identity (DenseMatrix<T> &A)
```

Parameters

in	*A*	reference to a DenseMatrix that shall be filled with entries

Example:

```
hdnum::DenseMatrix<double> A(4,4);
identity(A);
A.scientific(false); // fixed point representation for all DenseMatrix objects
A.width(10);
A.precision(5);
std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0          1          2          3
0     1.00000    0.00000    0.00000    0.00000
1     0.00000    1.00000    0.00000    0.00000
2     0.00000    0.00000    1.00000    0.00000
3     0.00000    0.00000    0.00000    1.00000
```

4.7.3.2 readMatrixFromFileDat()

```
template<typename REAL >
void readMatrixFromFileDat (
            const std::string & filename,
            DenseMatrix< REAL > & A )  [related]
```

Read matrix from a text file.

Parameters

in	*filename*	name of the text file
in,out	*A*	reference to a DenseMatrix

Example:

```
hdnum::DenseMatrix<number> L;
readMatrixFromFile("matrixL.dat", L );
std::cout « "L=" « L « std::endl;
```

Output:

```
Contents of "matrixL.dat":
1.000e+00  0.000e+00  0.000e+00
2.000e+00  1.000e+00  0.000e+00
3.000e+00  2.000e+00  1.000e+00

would give:
L=
0          1          2
0   1.000e+00  0.000e+00  0.000e+00
1   2.000e+00  1.000e+00  0.000e+00
2   3.000e+00  2.000e+00  1.000e+00
```

4.7.3.3 readMatrixFromFileMatrixMarket()

```
template<typename REAL >
void readMatrixFromFileMatrixMarket (
            const std::string & filename,
            DenseMatrix< REAL > & A )  [related]
```

Read matrix from a matrix market file.

Parameters

in	*filename*	name of the text file
in,out	*A*	reference to a DenseMatrix

Example:

```
hdnum::DenseMatrix<number> L;
readMatrixFromFile("matrixL.mtx", L );
std::cout « "L=" « L « std::endl;
```

Output:

```
Contents of "matrixL.mtx":
3 3 6
1 1 1
2 1 2
2 2 1
3 1 3
3 2 2
3 3 1

would give:
L=
0          1          2
```

```
0   1.000e+00  0.000e+00  0.000e+00
1   2.000e+00  1.000e+00  0.000e+00
2   3.000e+00  2.000e+00  1.000e+00
```

4.7.3.4 spd()

```
template<typename REAL >
void spd (
            DenseMatrix< REAL > & A )  [related]
```

Function: make a symmetric and positive definite matrix

```
template<typename REAL>
inline void spd (DenseMatrix<REAL> &A)
```

Parameters

in	*A*	reference to a DenseMatrix that shall be filled with entries

Example:

```
hdnum::DenseMatrix<double> A(4,4);
spd(A);
A.scientific(false); // fixed point representation for all DenseMatrix objects
A.width(10);
A.precision(5);
std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0          1          2          3
0     4.00000   -1.00000   -0.25000   -0.11111
1    -1.00000    4.00000   -1.00000   -0.25000
2    -0.25000   -1.00000    4.00000   -1.00000
3    -0.11111   -0.25000   -1.00000    4.00000
```

4.7.3.5 vandermonde()

```
template<typename REAL >
void vandermonde (
            DenseMatrix< REAL > & A,
            const Vector< REAL > x )  [related]
```

Function: make a vandermonde matrix

```
template<typename REAL>
inline void vandermonde (DenseMatrix<REAL> &A, const Vector<REAL> x)
```

Parameters

in	*A*	reference to a DenseMatrix that shall be filled with entries
in	*x*	constant reference to a Vector

Example:

```
hdnum::Vector<double> x(4);
fill(x,2.0,1.0);
hdnum::DenseMatrix<double> A(4,4);
vandermonde(A,x);
A.scientific(false); // fixed point representation for all DenseMatrix objects
A.width(10);
A.precision(5);
x.scientific(false); // fixed point representation for all Vector objects
x.width(10);
x.precision(5);
std::cout « "x=" « x « std::endl;
std::cout « "A=" « A « std::endl;
```

Output:

```
x=
[ 0]   2.00000
[ 1]   3.00000
[ 2]   4.00000
[ 3]   5.00000

A=
0          1          2          3
0     1.00000    2.00000    4.00000    8.00000
1     1.00000    3.00000    9.00000   27.00000
2     1.00000    4.00000   16.00000   64.00000
3     1.00000    5.00000   25.00000  125.00000
```

The documentation for this class was generated from the following file:

- src/densematrix.hh

4.8 hdnum::DIRK< M, S > Class Template Reference

Implementation of a general Diagonal Implicit Runge-Kutta method.

```
#include <ode.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type
- typedef DenseMatrix< number_type > ButcherTableau

  the type of a Butcher tableau

Public Member Functions

- DIRK (const M &model_, const S &newton_, const ButcherTableau &butcher_, const int order_)
- DIRK (const M &model_, const S &newton_, const std::string method)
- void set_dt (time_type dt_)

  set time step for subsequent steps
- void set_verbosity (size_type verbosity_)

  set verbosity level
- void step ()

  do one step
- bool get_error () const

  get current state
- void set_state (time_type t_, const Vector< number_type > &u_)

  set current state
- const Vector< number_type > & get_state () const

  get current state
- time_type get_time () const

  get current time
- time_type get_dt () const

  get dt used in last step (i.e. to compute current state)
- size_type get_order () const

  return consistency order of the method
- void get_info () const

  print some information

4.8.1 Detailed Description

template<class M, class S>
class hdnum::DIRK< M, S >

Implementation of a general Diagonal Implicit Runge-Kutta method.

The ODE solver is parametrized by a model. The model also exports all relevant types for time and states. The ODE solver encapsulates the states needed for the computation.

Template Parameters

M	the model type
S	nonlinear solver

4.8.2 Constructor & Destructor Documentation

4.8.2.1 DIRK() [1/2]

```
template<class M , class S >
hdnum::DIRK< M, S >::DIRK (
```

```
            const M & model_,
            const S & newton_,
            const ButcherTableau & butcher_,
            const int order_ )  [inline]
```

constructor stores reference to the model and requires a butcher tableau

4.8.2.2 DIRK() [2/2]

```
template<class M , class S >
hdnum::DIRK< M, S >::DIRK (
            const M & model_,
            const S & newton_,
            const std::string method )  [inline]
```

constructor stores reference to the model and sets the default butcher tableau corresponding to the given order

The documentation for this class was generated from the following file:

- src/ode.hh

4.9 hdnum::EE< M > Class Template Reference

Explicit Euler method as an example for an ODE solver.

```
#include <ode.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- EE (const M &model_)

  constructor stores reference to the model
- void set_dt (time_type dt_)

  set time step for subsequent steps
- void step ()

  do one step
- void set_state (time_type t_, const Vector< number_type > &u_)

  set current state
- const Vector< number_type > & get_state () const

  get current state
- time_type get_time () const

  get current time
- time_type get_dt () const

  get dt used in last step (i.e. to compute current state)
- size_type get_order () const

  return consistency order of the method

4.9.1 Detailed Description

template<class M>
class hdnum::EE< M >

Explicit Euler method as an example for an ODE solver.

The ODE solver is parametrized by a model. The model also exports all relevant types for time and states. The ODE solver encapsulates the states needed for the computation.

Template Parameters

M	the model type

The documentation for this class was generated from the following file:

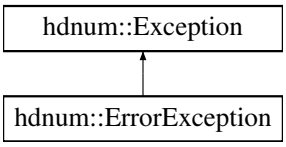
- src/ode.hh

4.10 hdnum::ErrorException Class Reference

General Error.

```
#include <exceptions.hh>
```

Inheritance diagram for hdnum::ErrorException:



Additional Inherited Members

4.10.1 Detailed Description

General Error.

The documentation for this class was generated from the following file:

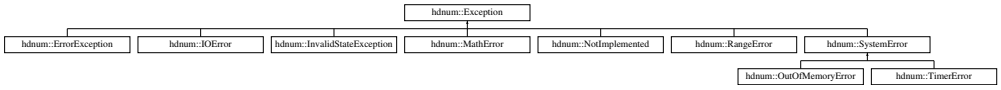
- src/exceptions.hh

4.11 hdnum::Exception Class Reference

Base class for Exceptions.

```
#include <exceptions.hh>
```

Inheritance diagram for hdnum::Exception:



Public Member Functions

- void message (const std::string &message)

  store string in internal message buffer
- const std::string & what () const

  output internal message buffer

4.11.1 Detailed Description

Base class for Exceptions.

all HDNUM exceptions are derived from this class via trivial subclassing:

```
class MyException : public Dune::Exception {};
```

You should not `throw` a Dune::Exception directly but use the macro DUNE_THROW() instead which fills the message-buffer of the exception in a standard way and features a way to pass the result in the operator<<-style

See also

HDNUM_THROW, IOError, MathError

The documentation for this class was generated from the following file:

- src/exceptions.hh

4.12 hdnum::GenericNonlinearProblem< Lambda, Vec > Class Template Reference

A generic problem class that can be set up with a lambda defining F(x)=0.

```
#include <newton.hh>
```

Public Types

- typedef std::size_t size_type

  export size_type
- typedef Vec::value_type number_type

  export number_type

Public Member Functions

- GenericNonlinearProblem (const Lambda &l_, const Vec &x_, number_type eps_=1e-7)

  constructor stores parameter lambda
- std::size_t size () const

  return number of componentes for the model
- void F (const Vec &x, Vec &result) const

  model evaluation
- void F_x (const Vec &x, DenseMatrix< number_type > &result) const

  jacobian evaluation needed for implicit solvers

4.12.1 Detailed Description

template< typename Lambda, typename Vec >
class hdnum::GenericNonlinearProblem< Lambda, Vec >

A generic problem class that can be set up with a lambda defining F(x)=0.

Template Parameters

Lambda	mapping a Vector to a Vector
Vec	the type for the Vector

The documentation for this class was generated from the following file:

- src/newton.hh

4.13 hdnum::Heun2< M > Class Template Reference

Heun method (order 2 with 2 stages)

```
#include <ode.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- Heun2 (const M &model_)

  constructor stores reference to the model
- void set_dt (time_type dt_)

  set time step for subsequent steps
- void step ()

  do one step
- void set_state (time_type t_, const Vector< number_type > &u_)

  set current state
- const Vector< number_type > & get_state () const

  get current state
- time_type get_time () const

  get current time
- time_type get_dt () const

  get dt used in last step (i.e. to compute current state)
- size_type get_order () const

  return consistency order of the method

4.13.1 Detailed Description

template<class M>
class hdnum::Heun2< M >

Heun method (order 2 with 2 stages)

The ODE solver is parametrized by a model. The model also exports all relevant types for time and states. The ODE solver encapsulates the states needed for the computation.

Template Parameters

M	the model type

The documentation for this class was generated from the following file:

- src/ode.hh

4.14 hdnum::Heun3< M > Class Template Reference

Heun method (order 3 with 3 stages)

```
#include <ode.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- Heun3 (const M &model_)

  constructor stores reference to the model
- void set_dt (time_type dt_)

  set time step for subsequent steps
- void step ()

  do one step
- void set_state (time_type t_, const Vector< number_type > &u_)

  set current state
- const Vector< number_type > & get_state () const

  get current state
- time_type get_time () const

  get current time
- time_type get_dt () const

  get dt used in last step (i.e. to compute current state)
- size_type get_order () const

  return consistency order of the method

4.14.1 Detailed Description

template<class M>
class hdnum::Heun3< M >

Heun method (order 3 with 3 stages)

The ODE solver is parametrized by a model. The model also exports all relevant types for time and states. The ODE solver encapsulates the states needed for the computation.

Template Parameters

M	the model type

The documentation for this class was generated from the following file:

- src/ode.hh

4.15 hdnum::IE< M, S > Class Template Reference

Implicit Euler using Newton's method to solve nonlinear system.

```
#include <ode.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- IE (const M &model_, const S &newton_)

  constructor stores reference to the model
- void set_dt (time_type dt_)

  set time step for subsequent steps
- void set_verbosity (size_type verbosity_)

  set verbosity level
- void step ()

  do one step
- bool get_error () const

  get current state
- void set_state (time_type t_, const Vector< number_type > &u_)

  set current state
- const Vector< number_type > & get_state () const

  get current state
- time_type get_time () const

  get current time
- time_type get_dt () const

  get dt used in last step (i.e. to compute current state)
- size_type get_order () const

  return consistency order of the method
- void get_info () const

  print some information

4.15.1 Detailed Description

template<class M, class S>
class hdnum::IE< M, S >

Implicit Euler using Newton's method to solve nonlinear system.

The ODE solver is parametrized by a model. The model also exports all relevant types for time and states. The ODE solver encapsulates the states needed for the computation.

Template Parameters

M	the model type
S	nonlinear solver

The documentation for this class was generated from the following file:

- src/ode.hh

4.16 hdnum::ImplicitRungeKuttaStepProblem< M > Class Template Reference

Nonlinear problem we need to solve to do one step of an implicit Runge Kutta method.

```
#include <rungekutta.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- ImplicitRungeKuttaStepProblem (const M &model_, DenseMatrix< number_type > A_, Vector< number_type > b_, Vector< number_type > c_, time_type t_, Vector< number_type > u_, time_type dt_)

  constructor stores parameter lambda
- std::size_t size () const

  return number of componentes for the model
- void F (const Vector< number_type > &x, Vector< number_type > &result) const

  model evaluation
- void F_x (const Vector< number_type > &x, DenseMatrix< number_type > &result) const

  jacobian evaluation needed for newton in implicite solvers

4.16.1 Detailed Description

template<class M>
class hdnum::ImplicitRungeKuttaStepProblem< M >

Nonlinear problem we need to solve to do one step of an implicit Runge Kutta method.

The documentation for this class was generated from the following file:

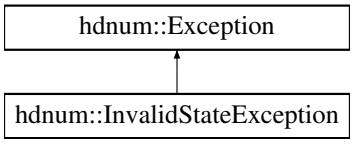
- src/rungekutta.hh

4.17 hdnum::InvalidStateException Class Reference

Default exception if a function was called while the object is not in a valid state for that function.

```
#include <exceptions.hh>
```

Inheritance diagram for hdnum::InvalidStateException:



Additional Inherited Members

4.17.1 Detailed Description

Default exception if a function was called while the object is not in a valid state for that function.

The documentation for this class was generated from the following file:

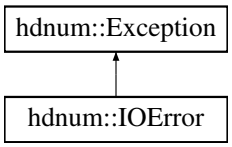
- src/exceptions.hh

4.18 hdnum::IOError Class Reference

Default exception class for I/O errors.

```
#include <exceptions.hh>
```

Inheritance diagram for hdnum::IOError:



Additional Inherited Members

4.18.1 Detailed Description

Default exception class for I/O errors.

This is a superclass for any errors dealing with file/socket I/O problems like

- file not found
- could not write file
- could not connect to remote socket

The documentation for this class was generated from the following file:

- src/exceptions.hh

4.19 hdnum::Kutta3< M > Class Template Reference

Kutta method (order 3 with 3 stages)

```
#include <ode.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- Kutta3 (const M &model_)

  constructor stores reference to the model
- void set_dt (time_type dt_)

  set time step for subsequent steps
- void step ()

  do one step
- void set_state (time_type t_, const Vector< number_type > &u_)

  set current state
- const Vector< number_type > & get_state () const

  get current state
- time_type get_time () const

  get current time
- time_type get_dt () const

  get dt used in last step (i.e. to compute current state)
- size_type get_order () const

  return consistency order of the method

4.19.1 Detailed Description

template<class M>
class hdnum::Kutta3< M >

Kutta method (order 3 with 3 stages)

The ODE solver is parametrized by a model. The model also exports all relevant types for time and states. The ODE solver encapsulates the states needed for the computation.

Template Parameters

M	the model type

The documentation for this class was generated from the following file:

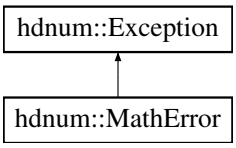
- src/ode.hh

4.20 hdnum::MathError Class Reference

Default exception class for mathematical errors.

```
#include <exceptions.hh>
```

Inheritance diagram for hdnum::MathError:



Additional Inherited Members

4.20.1 Detailed Description

Default exception class for mathematical errors.

This is the superclass for all errors which are caused by mathematical problems like

- matrix not invertible
- not convergent

The documentation for this class was generated from the following file:

- src/exceptions.hh

4.21 hdnum::ModifiedEuler< M > Class Template Reference

Modified Euler method (order 2 with 2 stages)

```
#include <ode.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- ModifiedEuler (const M &model_)

  constructor stores reference to the model
- void set_dt (time_type dt_)

  set time step for subsequent steps
- void step ()

  do one step
- void set_state (time_type t_, const Vector< number_type > &u_)

  set current state
- const Vector< number_type > & get_state () const

  get current state
- time_type get_time () const

  get current time
- time_type get_dt () const

  get dt used in last step (i.e. to compute current state)
- size_type get_order () const

  return consistency order of the method

4.21.1 Detailed Description

template<class M>
class hdnum::ModifiedEuler< M >

Modified Euler method (order 2 with 2 stages)

The ODE solver is parametrized by a model. The model also exports all relevant types for time and states. The ODE solver encapsulates the states needed for the computation.

Template Parameters

M	the model type

The documentation for this class was generated from the following file:

- src/ode.hh

4.22 hdnum::Newton Class Reference

Solve nonlinear problem using a damped Newton method.

```
#include <newton.hh>
```

Public Member Functions

- Newton ()

  constructor stores reference to the model
- void set_maxit (size_type n)

  maximum number of iterations before giving up
- void **set_sigma** (double sigma_)
- void set_linesearchsteps (size_type n)

  maximum number of steps in linesearch before giving up
- void set_verbosity (size_type n)

  control output given 0=nothing, 1=summary, 2=every step, 3=include line search
- void set_abslimit (double l)

  basolute limit for defect
- void set_reduction (double l)

  reduction factor
- template<class M >
  void solve (const M &model, Vector< typename M::number_type > &x) const

  do one step
- bool **has_converged** () const
- size_type **iterations** () const

4.22.1 Detailed Description

Solve nonlinear problem using a damped Newton method.

The Newton solver is parametrized by a model. The model also exports all relevant types for types.

The documentation for this class was generated from the following file:

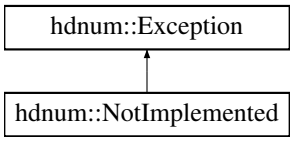
- src/newton.hh

4.23 hdnum::NotImplemented Class Reference

Default exception for dummy implementations.

```
#include <exceptions.hh>
```

Inheritance diagram for hdnum::NotImplemented:



Additional Inherited Members

4.23.1 Detailed Description

Default exception for dummy implementations.

This exception can be used for functions/methods

- that have to be implemented but should never be called
- that are missing

The documentation for this class was generated from the following file:

- src/exceptions.hh

4.24 hdnum::oc::OpCounter< F > Class Template Reference

```
#include <opcounter.hh>
```

Classes

- struct Counters

  Struct storing the number of operations.

Public Types

- using **size_type** = std::size_t
- using **value_type** = F

Public Member Functions

- template<typename T >
  OpCounter (const T &t, typename std::enable_if< std::is_same< T, int >::value and !std::is_same< F, int >::value >::type *=nullptr)
- **OpCounter** (const F &f)
- **OpCounter** (F &&f)
- **OpCounter** (const char *s)
- OpCounter & **operator=** (const char *s)
- **operator F** () const
- OpCounter & **operator=** (const F &f)
- OpCounter & **operator=** (F &&f)
- F * **data** ()
- const F * **data** () const

Static Public Member Functions

- static void **additions** (std::size_t n)
- static void **multiplications** (std::size_t n)
- static void **divisions** (std::size_t n)
- static void **reset** ()
- template<typename Stream >
  static void reportOperations (Stream &os, bool doReset=false)

  Report operations to stream object.
- static size_type totalOperationCount (bool doReset=false)

  Return total number of operations.

Public Attributes

- F **_v**

Static Public Attributes

- static Counters **counters**

Friends

- std::ostream & **operator**<< (std::ostream &os, const OpCounter &f)
- std::istringstream & **operator**>> (std::istringstream &iss, OpCounter &f)

4.24.1 Detailed Description

template<typename F>
class hdnum::oc::OpCounter< F >

Class counting operations

This is done by overloading operations and storing the numbers in a static class member.

The documentation for this class was generated from the following file:

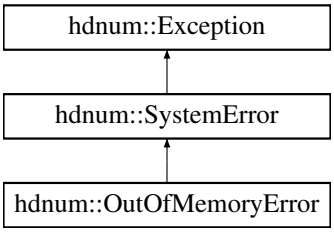
- src/opcounter.hh

4.25 hdnum::OutOfMemoryError Class Reference

Default exception if memory allocation fails.

```
#include <exceptions.hh>
```

Inheritance diagram for hdnum::OutOfMemoryError:



Additional Inherited Members

4.25.1 Detailed Description

Default exception if memory allocation fails.

The documentation for this class was generated from the following file:

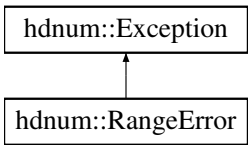
- src/exceptions.hh

4.26 hdnum::RangeError Class Reference

Default exception class for range errors.

```
#include <exceptions.hh>
```

Inheritance diagram for hdnum::RangeError:



Additional Inherited Members

4.26.1 Detailed Description

Default exception class for range errors.

This is the superclass for all errors which are caused because the user tries to access data that was not allocated before. These can be problems like

- accessing array entries behind the last entry
- adding the fourth non zero entry in a sparse matrix with only three non zero entries per row

The documentation for this class was generated from the following file:

- src/exceptions.hh

4.27 hdnum::RE< M, S > Class Template Reference

Adaptive one-step method using Richardson extrapolation.

```
#include <ode.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- RE (const M &model_, S &solver_)

  constructor stores reference to the model
- void set_dt (time_type dt_)

  set time step for subsequent steps
- void set_TOL (time_type TOL_)

  set tolerance for adaptive computation
- void step ()

  do one step
- const Vector< number_type > & get_state () const

  get current state
- time_type get_time () const

  get current time
- time_type get_dt () const

  get dt used in last step (i.e. to compute current state)
- size_type get_order () const

  return consistency order of the method
- void get_info () const

  print some information

4.27.1 Detailed Description

template<class M, class S>
class hdnum::RE< M, S >

Adaptive one-step method using Richardson extrapolation.

Template Parameters

M	a model
S	any of the (non-adaptive) one step methods (solving model M)

The documentation for this class was generated from the following file:

- src/ode.hh

4.28 hdnum::RKF45< M > Class Template Reference

Adaptive Runge-Kutta-Fehlberg method.

```
#include <ode.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- RKF45 (const M &model_)

  constructor stores reference to the model
- void set_dt (time_type dt_)

  set time step for subsequent steps
- void set_TOL (time_type TOL_)

  set tolerance for adaptive computation
- void step ()

  do one step
- const Vector< number_type > & get_state () const

  get current state
- time_type get_time () const

  get current time
- time_type get_dt () const

  get dt used in last step (i.e. to compute current state)
- size_type get_order () const

  return consistency order of the method
- void get_info () const

  print some information

4.28.1 Detailed Description

template<class M>
class hdnum::RKF45< M >

Adaptive Runge-Kutta-Fehlberg method.

Template Parameters

M	the model type

The documentation for this class was generated from the following file:

- src/ode.hh

4.29 hdnum::SparseMatrix< REAL >::row_iterator Class Reference

Public Types

- using **self_type** = row_iterator
- using **difference_type** = std::ptrdiff_t
- using **value_type** = self_type
- using **pointer** = self_type *
- using **reference** = self_type &
- using **iterator_category** = std::random_access_iterator_tag

Public Member Functions

- **row_iterator** (std::vector< size_type >::iterator rowPtrIter, std::vector< size_type >::iterator colIndicesIter, typename std::vector< REAL >::iterator valIter)
- column_iterator **begin** ()
- column_iterator **end** ()
- column_index_iterator **ibegin** ()
- column_index_iterator **iend** ()
- self_type & **operator++** ()
- self_type **operator++** (int junk)
- self_type & **operator+=** (difference_type offset)
- self_type & **operator-=** (difference_type offset)
- self_type **operator-** (difference_type offset)
- self_type **operator+** (difference_type offset)
- reference **operator[ ]** (difference_type offset)
- bool **operator**< (const self_type &other)
- bool **operator**> (const self_type &other)
- self_type & **operator*** ()
- bool **operator==** (const self_type &rhs)
- bool **operator!=** (const self_type &rhs)

Friends

- self_type **operator+** (const difference_type &offset, const self_type &sec)

The documentation for this class was generated from the following file:

- src/sparsematrix.hh

4.30 hdnum::RungeKutta< M, S > Class Template Reference

classical Runge-Kutta method (order n with n stages)

```
#include <rungekutta.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- RungeKutta (const M &model_, DenseMatrix< number_type > A_, Vector< number_type > b_, Vector< number_type > c_)

  constructor stores reference to the model
- RungeKutta (const M &model_, DenseMatrix< number_type > A_, Vector< number_type > b_, Vector< number_type > c_, number_type sigma_)

  constructor stores reference to the model
- void set_dt (time_type dt_)

  set time step for subsequent steps
- bool check_explicit ()

  test if method is explicit
- void step ()

  do one step
- void set_state (time_type t_, const Vector< number_type > &u_)

  set current state
- const Vector< number_type > & get_state () const

  get current state
- time_type get_time () const

  get current time
- time_type get_dt () const

  get dt used in last step (i.e. to compute current state)
- void set_verbosity (int verbosity_)

  how much should the ODE solver talk

4.30.1 Detailed Description

template<class M, class S = Newton>
class hdnum::RungeKutta< M, S >

classical Runge-Kutta method (order n with n stages)

The ODE solver is parametrized by a model. The model also exports all relevant types for time and states. The ODE solver encapsulates the states needed for the computation.

Template Parameters

M	The model type
S	(Nonlinear) solver (default is Newton)

The documentation for this class was generated from the following file:

- src/rungekutta.hh

4.31 hdnum::RungeKutta4< M > Class Template Reference

classical Runge-Kutta method (order 4 with 4 stages)

```
#include <ode.hh>
```

Public Types

- typedef M::size_type size_type
  export size_type
- typedef M::time_type time_type
  export time_type
- typedef M::number_type number_type
  export number_type

Public Member Functions

- RungeKutta4 (const M &model_)
  constructor stores reference to the model
- void set_dt (time_type dt_)
  set time step for subsequent steps
- void step ()
  do one step
- void set_state (time_type t_, const Vector< number_type > &u_)
  set current state
- const Vector< number_type > & get_state () const
  get current state
- time_type get_time () const
  get current time
- time_type get_dt () const
  get dt used in last step (i.e. to compute current state)
- size_type get_order () const
  return consistency order of the method

4.31.1 Detailed Description

template<class M>
class hdnum::RungeKutta4< M >

classical Runge-Kutta method (order 4 with 4 stages)

The ODE solver is parametrized by a model. The model also exports all relevant types for time and states. The ODE solver encapsulates the states needed for the computation.

Template Parameters

M	the model type

The documentation for this class was generated from the following file:

- src/ode.hh

4.32 hdnum::SGrid< N, DF, dimension > Class Template Reference

Structured Grid for Finite Differences.

```
#include <sgrid.hh>
```

Public Types

- enum { **dim** = dimension }
- typedef std::size_t size_type

  Export size type.
- typedef N number_type

  Export number type.
- typedef DF DomainFunction

  Type of the function defining the domain.

Public Member Functions

- SGrid (const Vector< number_type > extent_, const Vector< size_type > size_, const DomainFunction &df_)

  Constructor.
- size_type getNeighborIndex (const size_type In, const size_type n_dim, const int n_side, const int k=1) const

  Provides the index of the k-th neighbor of the node with index In.
- bool isBoundaryNode (const size_type In) const

  Returns true if the node is on the boundary of the discrete compuational domain.
- size_type getNumberOfNodes () const

  Returns the number of nodes which are in the compuational domain.
- Vector< size_type > **getGridSize** () const
- Vector< number_type > getCellWidth () const

  Returns the cell width h of the structured grid.
- Vector< number_type > getCoordinates (const size_type In) const

  Returns the world coordinates of the node with the given node index.
- std::vector< Vector< number_type > > **getNodeCoordinates** () const

Public Attributes

- const size_type invalid_node

  The value which is returned to indicate an invalid node.

Static Public Attributes

- static const int positive = 1

  Side definitions for usage in getNeighborIndex(..)
- static const int **negative** = -1

4.32.1 Detailed Description

template<class N, class DF, int dimension>
class hdnum::SGrid< N, DF, dimension >

Structured Grid for Finite Differences.

Template Parameters

N	A continuous type representing coordinate values.
DF	A boolean function which defines the domain.
dimension	The grid dimension.

4.32.2 Constructor & Destructor Documentation

4.32.2.1 SGrid()

```
template<class N , class DF , int dimension>
hdnum::SGrid< N, DF, dimension >::SGrid (
            const Vector< number_type > extent_,
            const Vector< size_type > size_,
            const DomainFunction & df_ )  [inline]
```

Constructor.

Parameters

in	*extent_*	The extent of the grid domain. The actual computational domain may be smaller and is defined by the domain function df_.
in	*size_*	The number of nodes in each grid dimension.
in	*df_*	The domain function. It has to provide a boolean function evaluate(Vector<number_type> x) which returns true if the node which is positioned at the coordinates of x is within the computational domain.

4.32.3 Member Function Documentation

4.32.3.1 getNeighborIndex()

```
template<class N , class DF , int dimension>
size_type hdnum::SGrid< N, DF, dimension >::getNeighborIndex (
            const size_type ln,
            const size_type n_dim,
            const int n_side,
            const int k = 1 ) const  [inline]
```

Provides the index of the k-th neighbor of the node with index ln.

Parameters

in	*ln*	Index of the node whose neighbor is to be determined.
in	*n_dim*	The axes which connects the node and its neighbor (e.g. n_dim = 0 for a neighbor in the direction of the x-axes
in	*n_side*	Determines whether the neighbor is in positive of negative direction of the given axes. Should be either SGrid::positive or SGrid::negative .
in	*k*	For k=1 it will return the direct neighbor. Higher values will give distant nodes in the given direction. If the indicated node is not within the grid any more, then invalid_node will be returned. For k=0 it will simply return ln.

Returns

size_type The index of the neighbor node.

The documentation for this class was generated from the following file:

- src/sgrid.hh

4.33 hdnum::SparseMatrix< REAL > Class Template Reference

Sparse matrix Class with mathematical matrix operations.

```
#include <sparsematrix.hh>
```

Classes

- class builder
- class column_index_iterator
- class const_column_index_iterator
- class const_row_iterator
- class row_iterator

Public Types

- using size_type = std::size_t

  Types used for array indices.
- using column_iterator = typename std::vector< REAL >::iterator

  type of a regular column iterator (no access to indices)
- using const_column_iterator = typename std::vector< REAL >::const_iterator

  type of a const regular column iterator (no access to indices)

Public Member Functions

- SparseMatrix ()=default

  default constructor (empty SparseMatrix)
- SparseMatrix (const size_type _rows, const size_type _cols)

  constructor with added dimensions and columns
- size_type rowsize () const

  get number of rows of the matrix
- size_type colsize () const

  get number of columns of the matrix
- bool scientific () const

  pretty-print output properties
- row_iterator begin ()

  get a (possibly modifying) row iterator for the sparse matrix
- row_iterator end ()

  get a (possibly modifying) row iterator for the sparse matrix
- const_row_iterator cbegin () const

  get a (non modifying) row iterator for the sparse matrix
- const_row_iterator cend () const

  get a (non modifying) row iterator for the sparse matrix
- const_row_iterator begin () const
- const_row_iterator end () const
- void scientific (bool b) const

  Switch between floating point (default=true) and fixed point (false) display.
- size_type iwidth () const

  get index field width for pretty-printing
- size_type width () const

  get data field width for pretty-printing
- size_type precision () const

  get data precision for pretty-printing
- void iwidth (size_type i) const

  set index field width for pretty-printing
- void width (size_type i) const

  set data field width for pretty-printing
- void precision (size_type i) const

  set data precision for pretty-printing
- column_iterator **find** (const size_type row_index, const size_type col_index) const
- bool **exists** (const size_type row_index, const size_type col_index) const
- REAL & get (const size_type row_index, const size_type col_index)

  write access on matrix element A_ij using A.get(i,j)
- const REAL & operator() (const size_type row_index, const size_type col_index) const

  read-access on matrix element A_ij using A(i,j)
- bool operator== (const SparseMatrix &other) const

  checks whether two matricies are equal based on values and dimension
- bool operator!= (const SparseMatrix &other) const

  checks whether two matricies are unequal based on values and dimension
- bool **operator**< (const SparseMatrix &other)=delete
- bool **operator**> (const SparseMatrix &other)=delete
- bool **operator**<**=** (const SparseMatrix &other)=delete
- bool **operator**>**=** (const SparseMatrix &other)=delete
- SparseMatrix **transpose** () const
- SparseMatrix operator∗= (const REAL scalar)

- SparseMatrix operator/= (const REAL scalar)
- template<class V >
  void mv (Vector< V > &result, const Vector< V > &x) const

  matrix vector product y = A∗x
- Vector< REAL > operator∗ (const Vector< REAL > &x) const

  matrix vector product A∗x
- template<class V >
  void umv (Vector< V > &result, const Vector< V > &x) const

  update matrix vector product y += A∗x
- auto norm_infty () const

  calculate row sum norm
- std::string **to_string** () const noexcept
- void **print** () const noexcept
- SparseMatrix< REAL > matchingIdentity () const

  creates a matching identity

Static Public Member Functions

- static SparseMatrix identity (const size_type dimN)

  identity for the matrix

Related Functions

(Note that these are not member functions.)

- template<class REAL >
  void identity (SparseMatrix< REAL > &A)

4.33.1 Detailed Description

template<typename REAL>
class hdnum::SparseMatrix< REAL >

Sparse matrix Class with mathematical matrix operations.

4.33.2 Constructor & Destructor Documentation

4.33.2.1 SparseMatrix()

```
template<typename REAL >
hdnum::SparseMatrix< REAL >::SparseMatrix ( )  [default]
```

default constructor (empty SparseMatrix)

Example:

```
hdnum::SparseMatrix<double> A();
auto nRows = A.rowsize();
std::cout « "Matrix A has " « nRows « " rows." « std::endl;
```

Output:

```
Matrix A has 0 rows.
```

4.33.3 Member Function Documentation

4.33.3.1 begin() [1/2]

```
template<typename REAL >
row_iterator hdnum::SparseMatrix< REAL >::begin ( ) [inline]
```

get a (possibly modifying) row iterator for the sparse matrix

The iterator points to the first row in the matrix.

Example:

```
// A is of type hdnum::SparseMatrix<int> and contains some values
// the deduced variable type for row_it is
// hdnum::SparseMatrix<int>::row_iterator
// but thats way to long to type out ;)
for(auto row_it = A.begin(); row_it != A.end(); row_it++) {
    for(auto val_it = row_it.begin(); val_it != row_it.end(); val_it++) {
        *val_it = 1;
    }
}
```

4.33.3.2 begin() [2/2]

```
template<typename REAL >
const_row_iterator hdnum::SparseMatrix< REAL >::begin ( ) const [inline]
```

See also

cbegin() const

4.33.3.3 cbegin()

```
template<typename REAL >
const_row_iterator hdnum::SparseMatrix< REAL >::cbegin ( ) const [inline]
```

get a (non modifying) row iterator for the sparse matrix

The iterator points to the first row in the matrix.

4.33.3.4 cend()

```
template<typename REAL >
const_row_iterator hdnum::SparseMatrix< REAL >::cend ( ) const [inline]
```

get a (non modifying) row iterator for the sparse matrix

The iterator points to the row one after the last one.

4.33.3.5 colsize()

```
template<typename REAL >
size_type hdnum::SparseMatrix< REAL >::colsize ( ) const  [inline]
```

get number of columns of the matrix

Example:

```
hdnum::SparseMatrix<double> A(4,5);
auto nRows = A.colsize();
std::cout « "Matrix A has " « nRows « " rows." « std::endl;
```

Output:

```
Matrix A has 4 rows.
```

4.33.3.6 end() [1/2]

```
template<typename REAL >
row_iterator hdnum::SparseMatrix< REAL >::end ( )  [inline]
```

get a (possibly modifying) row iterator for the sparse matrix

The iterator points to the row one after the last one.

Example:

```
// A is of type hdnum::SparseMatrix<int> and contains some values
// the deduced variable type for row_it is
// hdnum::SparseMatrix<int>::row_iterator
// but thats way to long to type out ;)
for(auto row_it = A.begin(); row_it != A.end(); row_it++) {
    for(auto val_it = row_it.begin(); val_it != row_it.end(); val_it++) {
        *val_it = 1;
    }
}
```

4.33.3.7 end() [2/2]

```
template<typename REAL >
const_row_iterator hdnum::SparseMatrix< REAL >::end ( ) const  [inline]
```

See also

cend() const

4.33.3.8 identity()

```
template<typename REAL >
static SparseMatrix hdnum::SparseMatrix< REAL >::identity (
            const size_type dimN )  [inline], [static]
```

identity for the matrix

Example:

```
auto A = hdnum::SparseMatrix<double>::identity(4);
// fixed point representation for all SparseMatrix objects
A.scientific(false);
A.width(8);
A.precision(3);
std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0        1        2        3
0     1.000    0.000    0.000    0.000
1     0.000    1.000    0.000    0.000
2     0.000    0.000    1.000    0.000
3     0.000    0.000    0.000    1.000
```

4.33.3.9 matchingIdentity()

```
template<typename REAL >
SparseMatrix<REAL> hdnum::SparseMatrix< REAL >::matchingIdentity ( ) const  [inline]
```

creates a matching identity

Example:

```
auto A = hdnum::SparseMatrix<double>(4, 5);
auto B = A.matchingIdentity();
// fixed point representation for all SparseMatrix objects
A.scientific(false);
A.width(8);
A.precision(3);
std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0        1        2        3
0     1.000    0.000    0.000    0.000
1     0.000    1.000    0.000    0.000
2     0.000    0.000    1.000    0.000
3     0.000    0.000    0.000    1.000
```

4.33.3.10 mv()

```
template<typename REAL >
template<class V >
void hdnum::SparseMatrix< REAL >::mv (
            Vector< V > & result,
            const Vector< V > & x ) const  [inline]
```

matrix vector product y = A∗x

Implements y = A∗x where x and y are a vectors and A is a matrix

Parameters

in	*result*	reference to the resulting Vector
in	*x*	constant reference to a Vector

4.33.3.11 norm_infty()

```
template<typename REAL >
auto hdnum::SparseMatrix< REAL >::norm_infty ( ) const  [inline]
```

calculate row sum norm

$$||A||_\infty = max_{i=1...m} \sum_{j=1}^{n} |a_{ij}|$$

4.33.3.12 operator∗()

```
template<typename REAL >
Vector<REAL> hdnum::SparseMatrix< REAL >::operator* (
            const Vector< REAL > & x ) const  [inline]
```

matrix vector product A∗x

Implements A∗x where x is a vectors and A is a matrix

Parameters

in	*x*	constant reference to a Vector

4.33.3.13 operator∗=()

```
template<typename REAL >
SparseMatrix hdnum::SparseMatrix< REAL >::operator*= (
            const REAL scalar )  [inline]
```

Element-wise multiplication of the matrix

Parameters

in	*scalar*	with same type as the matrix elements

4.33.3.14 operator/=()

```
template<typename REAL >
SparseMatrix hdnum::SparseMatrix< REAL >::operator/= (
            const REAL scalar )  [inline]
```

Element-wise division of the matrix

Parameters

in	*scalar*	with same type as the matrix elements

4.33.3.15 rowsize()

```
template<typename REAL >
size_type hdnum::SparseMatrix< REAL >::rowsize ( ) const  [inline]
```

get number of rows of the matrix

Example:

```
hdnum::SparseMatrix<double> A(4,5);
auto nRows = A.rowsize();
std::cout « "Matrix A has " « nRows « " rows." « std::endl;
```

Output:

```
Matrix A has 4 rows.
```

4.33.3.16 scientific()

```
template<typename REAL >
void hdnum::SparseMatrix< REAL >::scientific (
            bool b ) const  [inline]
```

Switch between floating point (default=true) and fixed point (false) display.

Example:

```
hdnum::SparseMatrix<double> A(4,4);
// fixed point representation for all SparseMatrix objects objects
A.scientific(false);
A.width(8); A.precision(3); identity(A);
// Defines the identity matrix of the same dimension
std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0         1         2         3
0     1.000     0.000     0.000     0.000
1     0.000     1.000     0.000     0.000
2     0.000     0.000     1.000     0.000
3     0.000     0.000     0.000     1.000
```

4.33.3.17 umv()

```
template<typename REAL >
template<class V >
void hdnum::SparseMatrix< REAL >::umv (
            Vector< V > & result,
            const Vector< V > & x ) const  [inline]
```

update matrix vector product y += A∗x

Implements y += A∗x where x and y are a vectors and A is a matrix

Parameters

in	*result*	reference to the resulting Vector
in	*x*	constant reference to a Vector

4.33.4 Friends And Related Function Documentation

4.33.4.1 identity()

```
template<class REAL >
void identity (
            SparseMatrix< REAL > & A )  [related]
```

Function: make identity matrix

```
template<class T>
inline void identity (SparseMatrix<T> &A)
```

Parameters

in	*A*	reference to a SparseMatrix that shall be filled with entries

Example:

```
hdnum::SparseMatrix<double> A(4,4);
identity(A);
// fixed point representation for all DenseMatrix objects
A.scientific(false);
A.width(10);
A.precision(5);
std::cout « "A=" « A « std::endl;
```

Output:

```
A=
0          1          2          3
0     1.00000    0.00000    0.00000    0.00000
1     0.00000    1.00000    0.00000    0.00000
2     0.00000    0.00000    1.00000    0.00000
3     0.00000    0.00000    0.00000    1.00000
```

The documentation for this class was generated from the following files:

- src/densematrix.hh
- src/sparsematrix.hh

4.34 hdnum::SquareRootProblem< N > Class Template Reference

Example class for a nonlinear model F(x) = 0;.

```
#include <newton.hh>
```

Public Types

- typedef std::size_t size_type

  export size_type
- typedef N number_type

  export number_type

Public Member Functions

- SquareRootProblem (number_type a_)

  constructor stores parameter lambda
- std::size_t size () const

  return number of componentes for the model
- void F (const Vector< N > &x, Vector< N > &result) const

  model evaluation
- void F_x (const Vector< N > &x, DenseMatrix< N > &result) const

  jacobian evaluation needed for implicit solvers

4.34.1 Detailed Description

template<class N>
class hdnum::SquareRootProblem< N >

Example class for a nonlinear model F(x) = 0;.

This example solves F(x) = x∗x - a = 0

Template Parameters

N	a type representing x and F components

The documentation for this class was generated from the following file:

- src/newton.hh

4.35 hdnum::StationarySolver< M > Class Template Reference

Stationary problem solver. E.g. for elliptic problmes.

```
#include <pde.hh>
```

Public Types

- typedef M::size_type size_type

  export size_type
- typedef M::time_type time_type

  export time_type
- typedef M::number_type number_type

  export number_type

Public Member Functions

- StationarySolver (const M &model_)

  constructor stores reference to the model
- void solve ()

  do one step
- const Vector< number_type > & get_state () const

  get current state
- size_type get_order () const

  return consistency order of the method

4.35.1 Detailed Description

template<class M>
class hdnum::StationarySolver< M >

Stationary problem solver. E.g. for elliptic problmes.

The PDE solver is parametrized by a model. The model also exports all relevant types for the solution. The PDE solver encapsulates the states needed for the computation.

Template Parameters

M	the model type

The documentation for this class was generated from the following file:

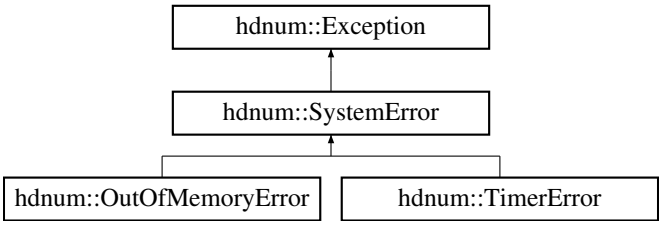
- src/pde.hh

4.36 hdnum::SystemError Class Reference

Default exception class for OS errors.

```
#include <exceptions.hh>
```

Inheritance diagram for hdnum::SystemError:



Additional Inherited Members

4.36.1 Detailed Description

Default exception class for OS errors.

This class is thrown when a system-call is used and returns an error.

The documentation for this class was generated from the following file:

- src/exceptions.hh

4.37 hdnum::Timer Class Reference

A simple stop watch.

```
#include <timer.hh>
```

Public Member Functions

- Timer ()

  A new timer, start immediately.
- void reset ()

  Reset timer.
- double elapsed () const

  Get elapsed user-time in seconds.

4.37.1 Detailed Description

A simple stop watch.

This class reports the elapsed user-time, i.e. time spent computing, after the last call to Timer::reset(). The results are seconds and fractional seconds. Note that the resolution of the timing depends on your OS kernel which should be somewhere in the milisecond range.

The class is basically a wrapper for the libc-function getrusage()

Taken from the DUNE project www.dune-project.org

The documentation for this class was generated from the following file:

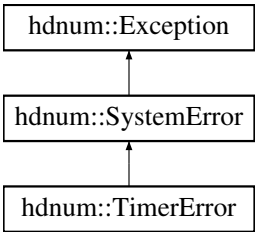
- src/timer.hh

4.38 hdnum::TimerError Class Reference

Exception thrown by the Timer class

```
#include <timer.hh>
```

Inheritance diagram for hdnum::TimerError:



Additional Inherited Members

4.38.1 Detailed Description

Exception thrown by the Timer class

The documentation for this class was generated from the following file:

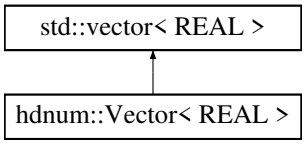
- src/timer.hh

4.39 hdnum::Vector< REAL > Class Template Reference

Class with mathematical vector operations.

```
#include <vector.hh>
```

Inheritance diagram for hdnum::Vector< REAL >:



Public Types

- typedef std::size_t size_type

  Type used for array indices.

Public Member Functions

- Vector ()

  default constructor, also inherited from the STL vector default constructor
- Vector (const size_t size, const REAL defaultvalue_=0)

  another constructor, with arguments, setting the default value for all entries of the vector of given size
- Vector (const std::initializer_list< REAL > &v)

  constructor from initializer list
- Vector & operator= (const REAL value)

  Assign all values of the Vector from one scalar value: x = value.
- Vector sub (size_type i, size_type m)

  Subvector extraction.
- Vector & operator*= (const REAL value)

  *Multiplication by a scalar value (x *= value)*
- Vector & operator/= (const REAL value)

  Division by a scalar value (x /= value)
- Vector & operator+= (const Vector &y)

  Add another vector (x += y)
- Vector & operator-= (const Vector &y)

  Subtract another vector (x -= y)
- Vector & update (const REAL alpha, const Vector &y)

  Update vector by addition of a scaled vector (x += a y)
- REAL operator* (Vector &x) const

  Inner product with another vector.
- Vector operator+ (Vector &x) const

  Adding two vectors x+y.
- Vector operator- (Vector &x) const

  vector subtraction x-y
- REAL two_norm_2 () const

  Square of the Euclidean norm.
- REAL two_norm () const

  Euclidean norm of a vector.
- bool scientific () const

  pretty-print output property: true = scientific, false = fixed point representation
- void scientific (bool b) const

  scientific(true) is the default, scientific(false) switches to the fixed point representation
- std::size_t iwidth () const

  get index field width for pretty-printing
- std::size_t width () const

  get data field width for pretty-printing
- std::size_t precision () const

  get data precision for pretty-printing
- void iwidth (std::size_t i) const

  set index field width for pretty-printing
- void width (std::size_t i) const

  set data field width for pretty-printing
- void precision (std::size_t i) const

  set data precision for pretty-printing

Related Functions

(Note that these are not member functions.)

- template<typename REAL >
  std::ostream & operator<< (std::ostream &os, const Vector< REAL > &x)

  Output operator for Vector.
- template<typename REAL >
  void gnuplot (const std::string &fname, const Vector< REAL > x)

  Output contents of a Vector x to a text file named fname.
- template<typename REAL >
  void readVectorFromFile (const std::string &filename, Vector< REAL > &vector)

  Read vector from a text file.
- template<class REAL >
  void fill (Vector< REAL > &x, const REAL &t, const REAL &dt)

  Fill vector, with entries starting at t, consecutively shifted by dt.
- template<class REAL >
  void unitvector (Vector< REAL > &x, std::size_t j)

  Defines j-th unitvector (j=0,...,n-1) where n = length of the vector.

4.39.1 Detailed Description

template<typename REAL>
class hdnum::Vector< REAL >

Class with mathematical vector operations.

4.39.2 Member Function Documentation

4.39.2.1 operator*()

```
template<typename REAL >
REAL hdnum::Vector< REAL >::operator* (
            Vector< REAL > & x ) const  [inline]
```

Inner product with another vector.

Example:

```
hdnum::Vector<double> x(2);
x.scientific(false); // set fixed point display mode
x[0] = 12.0;
x[1] = 3.0;
std::cout « "x=" « x « std::endl;
hdnum::Vector<double> y(2);
y[0] = 4.0;
y[1] = -1.0;
std::cout « "y=" « y « std::endl;
double s = x*y;
std::cout « "s = x*y = " « s « std::endl;
```

Output:

```
x=
[ 0]      12.0000000
[ 1]       3.0000000

y=
[ 0]       4.0000000
[ 1]      -1.0000000

s = x*y = 45.0000000
```

4.39.2.2 operator+()

```
template<typename REAL >
Vector hdnum::Vector< REAL >::operator+ (
            Vector< REAL > & x ) const  [inline]
```

Adding two vectors x+y.

Example:

```
hdnum::Vector<double> x(2);
x.scientific(false); // set fixed point display mode
x[0] = 12.0;
x[1] = 3.0;
std::cout « "x=" « x « std::endl;
hdnum::Vector<double> y(2);
y[0] = 4.0;
y[1] = -1.0;
std::cout « "y=" « y « std::endl;
std::cout « "x+y = " « x+y « std::endl;
```

Output:

```
x=
[ 0]      12.0000000
[ 1]       3.0000000

y=
[ 0]       4.0000000
[ 1]      -1.0000000

x+y =
[ 0]      16.0000000
[ 1]       2.0000000
```

4.39.2.3 operator-()

```
template<typename REAL >
Vector hdnum::Vector< REAL >::operator- (
            Vector< REAL > & x ) const  [inline]
```

vector subtraction x-y

Example:

```
hdnum::Vector<double> x(2);
x.scientific(false); // set fixed point display mode
x[0] = 12.0;
x[1] = 3.0;
std::cout « "x=" « x « std::endl;
hdnum::Vector<double> y(2);
y[0] = 4.0;
y[1] = -1.0;
std::cout « "y=" « y « std::endl;
std::cout « "x-y = " « x-y « std::endl;
```

Output:

```
x=
[ 0]      12.0000000
[ 1]       3.0000000

y=
[ 0]       4.0000000
[ 1]      -1.0000000

x-y =
[ 0]       8.0000000
[ 1]       4.0000000
```

4.39.2.4 operator=()

```
template<typename REAL >
Vector& hdnum::Vector< REAL >::operator= (
            const REAL value )  [inline]
```

Assign all values of the Vector from one scalar value: x = value.

Parameters

in	*value*	constant value which should be assigned

Example:

```
hdnum::Vector<double> x(4);
x = 1.23;
std::cout « "x=" « x « std::endl;
```

Output:

```
x=
[ 0]  1.2340000e+00
[ 1]  1.2340000e+00
[ 2]  1.2340000e+00
[ 3]  1.2340000e+00
```

4.39.2.5 scientific()

```
template<typename REAL >
void hdnum::Vector< REAL >::scientific (
            bool b ) const  [inline]
```

scientific(true) is the default, scientific(false) switches to the fixed point representation

Example:

```
hdnum::Vector<double> x(3);
x[0] = 2.0;
x[1] = 2.0;
x[2] = 1.0;
std::cout « "x=" « x « std::endl;
x.scientific(false); // set fixed point display mode
std::cout « "x=" « x « std::endl;
```

Output:

```
x=
[ 0]  2.0000000e+00
[ 1]  2.0000000e+00
[ 2]  1.0000000e+00

x=
[ 0]      2.0000000
[ 1]      2.0000000
[ 2]      1.0000000
```

4.39.2.6 sub()

```
template<typename REAL >
Vector hdnum::Vector< REAL >::sub (
            size_type i,
            size_type m )  [inline]
```

Subvector extraction.

Returns a new vector that is a subset of the components of the given vector.

Parameters

in	*i*	first index of the new vector
in	*m*	size of the new vector, i.e. it has components [i,i+m-1]

4.39.2.7 two_norm()

```
template<typename REAL >
REAL hdnum::Vector< REAL >::two_norm ( ) const  [inline]
```

Euclidean norm of a vector.

Example:

```
hdnum::Vector<double> x(3);
x.scientific(false); // set fixed point display mode
x[0] = 2.0;
x[1] = 2.0;
x[2] = 1.0;
std::cout « "x=" « x « std::endl;
std::cout « "euclidean norm of x = " « x.two_norm() « std::endl;
```

Output:

```
x=
[ 0]      2.0000000
[ 1]      2.0000000
[ 2]      1.0000000

euclidean norm of x = 3.0000000
```

4.39.3 Friends And Related Function Documentation

4.39.3.1 fill()

```
template<class REAL >
void fill (
            Vector< REAL > & x,
            const REAL & t,
            const REAL & dt )  [related]
```

Fill vector, with entries starting at t, consecutively shifted by dt.

Example:

```
hdnum::Vector<double> x(5);
fill(x,2.01,0.1);
x.scientific(false); // set fixed point display mode
std::cout « "x=" « x « std::endl;
```

Output:

```
x=
[ 0]      2.0100000
[ 1]      2.1100000
[ 2]      2.2100000
[ 3]      2.3100000
[ 4]      2.4100000
```

4.39.3.2 gnuplot()

```
template<typename REAL >
void gnuplot (
            const std::string & fname,
            const Vector< REAL > x )  [related]
```

Output contents of a Vector x to a text file named fname.

Example:

```
hdnum::Vector<double> x(5);
unitvector(x,3);
x.scientific(false); // set fixed point display mode
gnuplot("test.dat",x);
```

Output:

```
Contents of 'test.dat':
0      0.0000000
1      0.0000000
2      0.0000000
3      1.0000000
4      0.0000000
```

4.39.3.3 operator<<()

```
template<typename REAL >
std::ostream & operator<< (
            std::ostream & os,
            const Vector< REAL > & x )  [related]
```

Output operator for Vector.

Example:

```
hdnum::Vector<double> x(3);
x[0] = 2.0;
x[1] = 2.0;
x[2] = 1.0;
std::cout « "x=" « x « std::endl;
```

Output:

```
x=
[ 0]  2.0000000e+00
[ 1]  2.0000000e+00
[ 2]  1.0000000e+00
```

4.39.3.4 readVectorFromFile()

```
template<typename REAL >
void readVectorFromFile (
            const std::string & filename,
            Vector< REAL > & vector )  [related]
```

Read vector from a text file.

Parameters

in	*filename*	name of the text file
in,out	*vector*	reference to a Vector

Example:

```
hdnum::Vector<number> x;
readVectorFromFile("x.dat", x );
std::cout « "x=" « x « std::endl;
```

Output:

```
Contents of "x.dat":
1.0
2.0
3.0

would give:
x=
[ 0]  1.0000000e+00
[ 1]  2.0000000e+00
[ 2]  3.0000000e+00
```

4.39.3.5 unitvector()

```
template<class REAL >
void unitvector (
            Vector< REAL > & x,
            std::size_t j )  [related]
```

Defines j-th unitvector (j=0,...,n-1) where n = length of the vector.

Example:

```
hdnum::Vector<double> x(5);
unitvector(x,3);
x.scientific(false); // set fixed point display mode
std::cout « "x=" « x « std::endl;
```

Output:

```
x=
[ 0]      0.0000000
[ 1]      0.0000000
[ 2]      0.0000000
[ 3]      1.0000000
[ 4]      0.0000000
```

The documentation for this class was generated from the following file:

- src/vector.hh

Chapter 5

File Documentation

5.1 src/exceptions.hh File Reference

A few common exception classes.

```
#include <string>
#include <sstream>
```

Classes

- class hdnum::Exception

  Base class for Exceptions.
- class hdnum::IOError

  Default exception class for I/O errors.
- class hdnum::MathError

  Default exception class for mathematical errors.
- class hdnum::RangeError

  Default exception class for range errors.
- class hdnum::NotImplemented

  Default exception for dummy implementations.
- class hdnum::SystemError

  Default exception class for OS errors.
- class hdnum::OutOfMemoryError

  Default exception if memory allocation fails.
- class hdnum::InvalidStateException

  Default exception if a function was called while the object is not in a valid state for that function.
- class hdnum::ErrorException

  General Error.

Macros

- #define **THROWSPEC**(E) #E << ": "
- #define HDNUM_THROW(E, m)
- #define **HDNUM_ERROR**(m)

Functions

- std::ostream & **hdnum::operator**<< (std::ostream &stream, const Exception &e)

5.1.1 Detailed Description

A few common exception classes.

This file defines a common framework for generating exception subclasses and to throw them in a simple manner. Taken from the DUNE project www.dune-project.org

5.1.2 Macro Definition Documentation

5.1.2.1 HDNUM_ERROR

```
#define HDNUM_ERROR(
                m )
```

Value:

```
        do { hdnum::ErrorException th__ex; std::ostringstream th__out;              \
        th__out « THROWSPEC(hdnum::ErrorException) « m; \
        th__ex.message(th__out.str()); \
        std::cout « th__ex.what() « std::endl; \
        throw th__ex;                                \
  } while (0)
```

5.1.2.2 HDNUM_THROW

```
#define HDNUM_THROW(
                E,
                m )
```

Value:

```
        do { E th__ex; std::ostringstream th__out;                \
        th__out « THROWSPEC(E) « m; th__ex.message(th__out.str()); throw th__ex; \
  } while (0)
```

Macro to throw an exception

Parameters

E	exception class derived from Dune::Exception
m	reason for this exception in ostream-notation

Example:

```
    if (filehandle == 0)
DUNE_THROW(FileError, "Could not open " « filename « " for reading!")
```

DUNE_THROW automatically adds information about the exception thrown to the text. If DUNE_DEVEL_MODE is defined more detail about the function where the exception happened is included. This mode can be activated via the `-enable-dunedevel` switch of ./configure

5.2 src/lr.hh File Reference

This file implements LU decomposition.

```
#include "vector.hh"
#include "densematrix.hh"
```

Functions

- template<class T >
  void hdnum::lr (DenseMatrix< T > &A, Vector< std::size_t > &p)

  compute lr decomposition of A with first nonzero pivoting
- template<class T >
  T hdnum::abs (const T &t)

  our own abs class that works also for multiprecision types
- template<class T >
  void hdnum::lr_partialpivot (DenseMatrix< T > &A, Vector< std::size_t > &p)

  lr decomposition of A with column pivoting
- template<class T >
  void hdnum::lr_fullpivot (DenseMatrix< T > &A, Vector< std::size_t > &p, Vector< std::size_t > &q)

  lr decomposition of A with full pivoting
- template<class T >
  void hdnum::permute_forward (const Vector< std::size_t > &p, Vector< T > &b)

  apply permutations to a right hand side vector
- template<class T >
  void hdnum::permute_backward (const Vector< std::size_t > &q, Vector< T > &z)

  apply permutations to a solution vector
- template<class T >
  void hdnum::row_equilibrate (DenseMatrix< T > &A, Vector< T > &s)

  perform a row equilibration of a matrix; return scaling for later use
- template<class T >
  void hdnum::apply_equilibrate (Vector< T > &s, Vector< T > &b)

  apply row equilibration to right hand side vector
- template<class T >
  void hdnum::solveL (const DenseMatrix< T > &A, Vector< T > &x, const Vector< T > &b)

  Assume L = lower triangle of A with l_ii=1, solve L x = b.
- template<class T >
  void hdnum::solveR (const DenseMatrix< T > &A, Vector< T > &x, const Vector< T > &b)

  Assume R = upper triangle of A and solve R x = b.
- template<class T >
  void hdnum::linsolve (DenseMatrix< T > &A, Vector< T > &x, Vector< T > &b)

  a complete solver; Note A, x and b are modified!

5.2.1 Detailed Description

This file implements LU decomposition.

5.3 src/newton.hh File Reference

Newton's method with line search.

```
#include "lr.hh"
#include <type_traits>
```

Classes

- class hdnum::SquareRootProblem< N >

  Example class for a nonlinear model F(x) = 0;.
- class hdnum::GenericNonlinearProblem< Lambda, Vec >

  A generic problem class that can be set up with a lambda defining F(x)=0.
- class hdnum::Newton

  Solve nonlinear problem using a damped Newton method.
- class hdnum::Banach

  Solve nonlinear problem using a fixed point iteration.

Functions

- template<typename F , typename X >
  GenericNonlinearProblem< F, X > hdnum::getNonlinearProblem (const F &f, const X &x, typename X↩ ::value_type eps=1e-7)

  A function returning a problem class.

5.3.1 Detailed Description

Newton's method with line search.

5.3.2 Function Documentation

5.3.2.1 getNonlinearProblem()

```
template<typename F , typename X >
GenericNonlinearProblem<F,X> hdnum::getNonlinearProblem (
            const F & f,
            const X & x,
            typename X::value_type eps = 1e-7 )
```

A function returning a problem class.

Automatic template parameter extraction makes fiddling with types unnecessary.

Template Parameters

F	a lambda mapping a Vector to a Vector
X	the type for the Vector

5.4 src/ode.hh File Reference

solvers for ordinary differential equations

```
#include <vector>
#include "newton.hh"
```

Classes

- class hdnum::EE< M >

  Explicit Euler method as an example for an ODE solver.
- class hdnum::ModifiedEuler< M >

  Modified Euler method (order 2 with 2 stages)
- class hdnum::Heun2< M >

  Heun method (order 2 with 2 stages)
- class hdnum::Heun3< M >

  Heun method (order 3 with 3 stages)
- class hdnum::Kutta3< M >

  Kutta method (order 3 with 3 stages)
- class hdnum::RungeKutta4< M >

  classical Runge-Kutta method (order 4 with 4 stages)
- class hdnum::RKF45< M >

  Adaptive Runge-Kutta-Fehlberg method.
- class hdnum::RE< M, S >

  Adaptive one-step method using Richardson extrapolation.
- class hdnum::IE< M, S >

  Implicit Euler using Newton's method to solve nonlinear system.
- class hdnum::DIRK< M, S >

  Implementation of a general Diagonal Implicit Runge-Kutta method.

Functions

- template<class T , class N >
  void hdnum::gnuplot (const std::string &fname, const std::vector< T > t, const std::vector< Vector< N > > u)

  gnuplot output for time and state sequence
- template<class T , class N >
  void hdnum::gnuplot (const std::string &fname, const std::vector< T > t, const std::vector< Vector< N > > u, const std::vector< T > dt)

  gnuplot output for time and state sequence

5.4.1 Detailed Description

solvers for ordinary differential equations

5.5 src/opcounter.hh File Reference

This file implements an operator counting class.

```
#include <type_traits>
#include <iostream>
#include <cmath>
#include <cstdlib>
```

Classes

- class hdnum::oc::OpCounter< F >
- struct hdnum::oc::OpCounter< F >::Counters

  Struct storing the number of operations.

Functions

- template<typename F >
  OpCounter< F > **hdnum::oc::operator-** (const OpCounter< F > &a)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator+** (const OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator+** (const OpCounter< F > &a, const F &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator+** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > >::type **hdnum::oc::operator+** (const OpCounter< F > &a, const T &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > >::type **hdnum::oc::operator+** (const T &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > & **hdnum::oc::operator+=** (OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > & **hdnum::oc::operator+=** (OpCounter< F > &a, const F &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > & >::type **hdnum::oc::operator+=** (OpCounter< F > &a, const T &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator-** (const OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator-** (const OpCounter< F > &a, const F &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator-** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > >::type **hdnum::oc::operator-** (const OpCounter< F > &a, const T &b)

- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > >::type **hdnum::oc::operator-** (const T &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > & **hdnum::oc::operator-=** (OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > & **hdnum::oc::operator-=** (OpCounter< F > &a, const F &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > & >::type **hdnum::oc::operator-=** (OpCounter< F > &a, const T &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator*** (const OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator*** (const OpCounter< F > &a, const F &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator*** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > >::type **hdnum::oc::operator*** (const OpCounter< F > &a, const T &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > >::type **hdnum::oc::operator*** (const T &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > & **hdnum::oc::operator*=** (OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > & **hdnum::oc::operator*=** (OpCounter< F > &a, const F &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > & >::type **hdnum::oc::operator*=** (OpCounter< F > &a, const T &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator/** (const OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator/** (const OpCounter< F > &a, const F &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::operator/** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > >::type **hdnum::oc::operator/** (const OpCounter< F > &a, const T &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > >::type **hdnum::oc::operator/** (const T &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > & **hdnum::oc::operator/=** (OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > & **hdnum::oc::operator/=** (OpCounter< F > &a, const F &b)
- template<typename F , typename T >
  std::enable_if< std::is_arithmetic< T >::value, OpCounter< F > & >::type **hdnum::oc::operator/=** (OpCounter< F > &a, const T &b)
- template<typename F >
  bool **hdnum::oc::operator<** (const OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator<** (const OpCounter< F > &a, const F &b)
- template<typename F >
  bool **hdnum::oc::operator<** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator<** (const OpCounter< F > &a, const T &b)

- template<typename F , typename T >
  bool **hdnum::oc::operator**< (const T &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator<=** (const OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator<=** (const OpCounter< F > &a, const F &b)
- template<typename F >
  bool **hdnum::oc::operator<=** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator<=** (const OpCounter< F > &a, const T &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator<=** (const T &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator>** (const OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator>** (const OpCounter< F > &a, const F &b)
- template<typename F >
  bool **hdnum::oc::operator>** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator>** (const OpCounter< F > &a, const T &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator>** (const T &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator>=** (const OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator>=** (const OpCounter< F > &a, const F &b)
- template<typename F >
  bool **hdnum::oc::operator>=** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator>=** (const OpCounter< F > &a, const T &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator>=** (const T &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator!=** (const OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator!=** (const OpCounter< F > &a, const F &b)
- template<typename F >
  bool **hdnum::oc::operator!=** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator!=** (const OpCounter< F > &a, const T &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator!=** (const T &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator==** (const OpCounter< F > &a, const OpCounter< F > &b)
- template<typename F >
  bool **hdnum::oc::operator==** (const OpCounter< F > &a, const F &b)
- template<typename F >
  bool **hdnum::oc::operator==** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator==** (const OpCounter< F > &a, const T &b)
- template<typename F , typename T >
  bool **hdnum::oc::operator==** (const T &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::exp** (const OpCounter< F > &a)
- template<typename F >
  OpCounter< F > **hdnum::oc::pow** (const OpCounter< F > &a, const OpCounter< F > &b)

- template<typename F >
  OpCounter< F > **hdnum::oc::pow** (const OpCounter< F > &a, const F &b)
- template<typename F , typename T >
  OpCounter< F > **hdnum::oc::pow** (const OpCounter< F > &a, const T &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::pow** (const F &a, const OpCounter< F > &b)
- template<typename F , typename T >
  OpCounter< F > **hdnum::oc::pow** (const T &a, const OpCounter< F > &b)
- template<typename F >
  OpCounter< F > **hdnum::oc::sin** (const OpCounter< F > &a)
- template<typename F >
  OpCounter< F > **hdnum::oc::cos** (const OpCounter< F > &a)
- template<typename F >
  OpCounter< F > **hdnum::oc::sqrt** (const OpCounter< F > &a)
- template<typename F >
  OpCounter< F > **hdnum::oc::abs** (const OpCounter< F > &a)

5.5.1 Detailed Description

This file implements an operator counting class.

5.6 src/pde.hh File Reference

solvers for partial differential equations

```
#include <vector>
#include "newton.hh"
```

Classes

- class hdnum::StationarySolver< M >

  Stationary problem solver. E.g. for elliptic problmes.

Functions

- template<class N , class G >
  void hdnum::pde_gnuplot2d (const std::string &fname, const Vector< N > solution, const G &grid)

  gnuplot output for stationary state

5.6.1 Detailed Description

solvers for partial differential equations

5.7 src/precision.hh File Reference

find machine precision for given float type

Functions

- template<typename X >
  int **hdnum::precision** (X &eps)

5.7.1 Detailed Description

find machine precision for given float type

5.8 src/qr.hh File Reference

This file implements QR decomposition using Gram-Schmidt method.

```
#include <cmath>
#include <utility>
#include "densematrix.hh"
#include "vector.hh"
```

Functions

- template<class T >
  DenseMatrix< T > hdnum::gram_schmidt (const DenseMatrix< T > &A)

  computes orthonormal basis of Im(A) using classical Gram-Schmidt
- template<class T >
  DenseMatrix< T > hdnum::modified_gram_schmidt (const DenseMatrix< T > &A)

  computes orthonormal basis of Im(A) using modified Gram-Schmidt
- template<class T >
  DenseMatrix< T > hdnum::qr_gram_schmidt_simple (DenseMatrix< T > &Q)

  computes qr decomposition using modified Gram-Schmidt - works only with small (m>n) and square matrices
- template<class T >
  DenseMatrix< T > hdnum::qr_gram_schmidt (DenseMatrix< T > &Q)

  computes qr decomposition using modified Gram-Schmidt - works only with small (m>n) and square matrices
- template<class T >
  DenseMatrix< T > hdnum::qr_gram_schmidt_pivoting (DenseMatrix< T > &Q, Vector< int > &p, int &rank, T threshold=0.00000000001)

  computes qr decomposition using modified Gram-Schmidt and pivoting - works with all types of matrices
- template<typename T >
  void hdnum::permute_forward (DenseMatrix< T > &A, Vector< int > &p)

  applies a permutation vector to a matrix

5.8.1 Detailed Description

This file implements QR decomposition using Gram-Schmidt method.

5.8.2 Function Documentation

5.8.2.1 gram_schmidt()

```
template<class T >
DenseMatrix<T> hdnum::gram_schmidt (
            const DenseMatrix< T > & A )
```

computes orthonormal basis of Im(A) using classical Gram-Schmidt

Template Parameters

hdnum::DenseMatrix<T>	A

Example:

```
hdnum::DenseMatrix<double> A({{2, 9},
                              {1, -5}});
hdnum::DenseMatrix<double> Q(hdnum::gram_schmidt(A));
std::cout « "A = " « A « std::endl;
std::cout « "Q = " « Q « std::endl;
```

Output:

```
A =
                  0          1
      0   2.000e+00  9.000e+00
      1   1.000e+00 -5.000e+00

Q =
                  0          1
      0   8.944e-01  4.472e-01
      1   4.472e-01 -8.944e-01
```

5.8.2.2 modified_gram_schmidt()

```
template<class T >
DenseMatrix<T> hdnum::modified_gram_schmidt (
            const DenseMatrix< T > & A )
```

computes orthonormal basis of Im(A) using modified Gram-Schmidt

Template Parameters

hdnum::DenseMatrix<T>	A

Example:

```
hdnum::DenseMatrix<double> A({{2, 9},
                              {1, -5}});
hdnum::DenseMatrix<double> Q(hdnum::modified_gram_schmidt(A));
std::cout « "A = " « A « std::endl;
std::cout « "Q = " « Q « std::endl;
```

Output:

```
A =
                  0          1
      0   2.000e+00  9.000e+00
```

```
      1   1.000e+00 -5.000e+00

Q =
                  0           1
      0   8.944e-01  4.472e-01
      1   4.472e-01 -8.944e-01
```

5.8.2.3 permute_forward()

```
template<typename T >
void hdnum::permute_forward (
            DenseMatrix< T > & A,
            Vector< int > & p )
```

applies a permutation vector to a matrix

Template Parameters

hdnum::DenseMatrix<T>	A

Parameters

hdnum::Vector<int>	p

Example:

```
hdnum::DenseMatrix<double> A({{2, 9},
                              {1, -5}});
hdnum::Vector<int> p({1, 0});
hdnum::permute_forward(A, p);
std::cout « "A = " « A « std::endl;
std::cout « "p = " « p « std::endl;
```

Output:

```
A =
                  0           1
      0   9.000e+00  2.000e+00
      1  -5.000e+00  1.000e+00

p =
        [ 0]               0
        [ 1]               1
```

5.8.2.4 qr_gram_schmidt()

```
template<class T >
DenseMatrix<T> hdnum::qr_gram_schmidt (
            DenseMatrix< T > & Q )
```

computes qr decomposition using modified Gram-Schmidt - works only with small (m>n) and square matrices

Template Parameters

hdnum::DenseMatrix<T>	Q

Example:

```
hdnum::DenseMatrix<double> A({{2, 9},
                              {1, -5}});
hdnum::DenseMatrix<double> Q(A);
hdnum::DenseMatrix<double> R(hdnum::qr_gram_schmidt(Q));
std::cout « "A = " « A « std::endl;
std::cout « "Q = " « Q « std::endl;
std::cout « "R = " « R « std::endl;
std::cout « "QR = " « Q*R « std::endl;
```

Output:

```
A =
                  0          1
      0   2.000e+00  9.000e+00
      1   1.000e+00 -5.000e+00

Q =
                  0          1
      0   8.944e-01  4.472e-01
      1   4.472e-01 -8.944e-01

R =
                  0          1
      0   2.236e+00  5.814e+00
      1   0.000e+00  8.497e+00

QR =
                  0          1
      0   2.000e+00  9.000e+00
      1   1.000e+00 -5.000e+00
```

5.8.2.5 qr_gram_schmidt_pivoting()

```
template<class T >
DenseMatrix<T> hdnum::qr_gram_schmidt_pivoting (
            DenseMatrix< T > & Q,
            Vector< int > & p,
            int & rank,
            T threshold = 0.00000000001 )
```

computes qr decomposition using modified Gram-Schmidt and pivoting - works with all types of matrices

Template Parameters

hdnum::DenseMatrix<T>	Q
T	threshold (optional)

Parameters

hdnum::Vector<int>	p
int	rank

Example:

```
hdnum::DenseMatrix<double> A({{5, 2, 3},
                              {11, 9, 2}});
hdnum::DenseMatrix<double> Q(A);
hdnum::Vector<int> p(A.colsize());
int rank;
hdnum::DenseMatrix<double> R(hdnum::qr_gram_schmidt_pivoting(Q, p, rank));
hdnum::DenseMatrix<double> Q_right_dimension(A.rowsize(), rank);
hdnum::DenseMatrix<double> R_right_dimension(rank, A.colsize());
for (int i = 0; i < Q_right_dimension.rowsize(); i++) {
    for (int j = 0; j < Q_right_dimension.colsize(); j++) {
        Q_right_dimension(i, j) = Q(i, j);
    }
}
for (int i = 0; i < R_right_dimension.rowsize(); i++) {
    for (int j = 0; j < R_right_dimension.colsize(); j++) {
        R_right_dimension(i, j) = R(i, j);
    }
}
hdnum::DenseMatrix<double> QR(Q_right_dimension*R_right_dimension);
hdnum::permute_forward(QR, p);
std::cout « "A = " « A « std::endl;
std::cout « "Q = " « Q_right_dimension « std::endl;
std::cout « "R = " « R_right_dimension « std::endl;
std::cout « "QR = " « QR « std::endl;
```

Output:

```
A =
                  0           1           2
      0   5.000e+00  2.000e+00  3.000e+00
      1   1.100e+01  9.000e+00  2.000e+00

Q =
                  0           1
      0   4.138e-01 -9.104e-01
      1   9.104e-01  4.138e-01

R =
                  0           1           2
      0   1.208e+01  9.021e+00  3.062e+00
      1   0.000e+00  1.903e+00 -1.903e+00

QR =
                  0           1           2
      0   5.000e+00  2.000e+00  3.000e+00
      1   1.100e+01  9.000e+00  2.000e+00
```

5.8.2.6 qr_gram_schmidt_simple()

```
template<class T >
DenseMatrix<T> hdnum::qr_gram_schmidt_simple (
            DenseMatrix< T > & Q )
```

computes qr decomposition using modified Gram-Schmidt - works only with small (m>n) and square matrices

Template Parameters

hdnum::DenseMatrix<T>	Q

Example:

```
hdnum::DenseMatrix<double> A({{2, 9},
                              {1, -5}});
hdnum::DenseMatrix<double> Q(A);
hdnum::DenseMatrix<double> R(hdnum::qr_gram_schmidt_simple(Q));
```

```
std::cout « "A = " « A « std::endl;
std::cout « "Q = " « Q « std::endl;
std::cout « "R = " « R « std::endl;
std::cout « "QR = " « Q*R « std::endl;
```

Output:

```
A =
                  0          1
      0   2.000e+00  9.000e+00
      1   1.000e+00 -5.000e+00

Q =
                  0          1
      0   8.944e-01  4.472e-01
      1   4.472e-01 -8.944e-01

R =
                  0          1
      0   2.236e+00  5.814e+00
      1   0.000e+00  8.497e+00

QR =
                  0          1
      0   2.000e+00  9.000e+00
      1   1.000e+00 -5.000e+00
```

5.9 src/qrhousholder.hh File Reference

This file implements QR decomposition using housholder transformation.

```
#include <cmath>
#include <cstdlib>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include "densematrix.hh"
#include "vector.hh"
```

Functions

- template<class REAL >
  DenseMatrix< REAL > **hdnum::creat_I_matrix** (size_t n)
- template<typename REAL >
  size_t hdnum::sgn (REAL val)

  Function that return the sign of a number.
- template<class REAL >
  void hdnum::qrhousholder (DenseMatrix< REAL > &A, hdnum::Vector< REAL > &v)

  Funktion that calculate the QR decoposition in place the elements of A will be replaced with the elements of v_{i}vectors and the upper diagonals elements of R and the diagonal elements of R will be saved in vectro v.
- template<class REAL >
  DenseMatrix< REAL > hdnum::qrhousholderexplizitQ (DenseMatrix< REAL > &A, hdnum::Vector< REAL > &v, bool show_Hi=false)

  Funktion that calculate the QR decoposition in place and return Q the elements of A will be replaced with the elements of v_{i}vectors and the upper diagonals elements of R and the diagonal elements of R will be saved in vectro v.

5.9.1 Detailed Description

This file implements QR decomposition using housholder transformation.

5.9.2 Function Documentation

5.9.2.1 qrhousholder()

```
template<class REAL >
void hdnum::qrhousholder (
            DenseMatrix< REAL > & A,
            hdnum::Vector< REAL > & v )
```

Funktion that calculate the QR decoposition in place the elements of A will be replaced with the elements of v_↩ {i}vectors and the upper diagonals elements of R and the diagonal elements of R will be saved in vectro v.

Template Parameters

A	the Matrix
v	oa vector of hdnum::Vector

5.9.2.2 qrhousholderexplizitQ()

```
template<class REAL >
DenseMatrix<REAL> hdnum::qrhousholderexplizitQ (
            DenseMatrix< REAL > & A,
            hdnum::Vector< REAL > & v,
            bool show_Hi = false )
```

Funktion that calculate the QR decoposition in place and return Q the elements of A will be replaced with the elements of v_{i}vectors and the upper diagonals elements of R and the diagonal elements of R will be saved in vectro v.

Template Parameters

A	the Matrix
v	oa vector of hdnum::Vector

Returns

Q matrix

5.10 src/rungekutta.hh File Reference

```
#include "vector.hh"
#include "newton.hh"
```

Classes

- class hdnum::ImplicitRungeKuttaStepProblem< M >

  Nonlinear problem we need to solve to do one step of an implicit Runge Kutta method.
- class hdnum::RungeKutta< M, S >

  classical Runge-Kutta method (order n with n stages)

Functions

- template<class M , class S >
  void hdnum::ordertest (const M &model, S solver, typename M::number_type T, typename M::number_type h_0, int l)

  Test convergence order of an ODE solver applied to a model problem.

5.10.1 Detailed Description

@general Runge-Kutta solver

5.10.2 Function Documentation

5.10.2.1 ordertest()

```
template<class M , class S >
void hdnum::ordertest (
            const M & model,
            S solver,
            typename M::number_type T,
            typename M::number_type h_0,
            int l )
```

Test convergence order of an ODE solver applied to a model problem.

Template Parameters

M	Type of model
S	Type of ODE solver

Parameters

model	Model problem
solver	ODE solver
T	Solve to time T
dt	Roughest time step size
l	Number of different time step sizes dt, dt/2, dt/4, ...

5.11 src/timer.hh File Reference

A simple timing class.

```
#include <sys/resource.h>
#include <ctime>
#include <cstring>
#include <cerrno>
#include "exceptions.hh"
```

Classes

- class hdnum::TimerError

  Exception thrown by the Timer class
- class hdnum::Timer

  A simple stop watch.

5.11.1 Detailed Description

A simple timing class.

Index

begin
hdnum::SparseMatrix< REAL >, 59

cbegin
hdnum::SparseMatrix< REAL >, 59
cend
hdnum::SparseMatrix< REAL >, 59
colsize
hdnum::DenseMatrix< REAL >, 14
hdnum::SparseMatrix< REAL >, 59

DIRK
hdnum::DIRK< M, S >, 32, 33

end
hdnum::SparseMatrix< REAL >, 60
exceptions.hh
HDNUM_ERROR, 78
HDNUM_THROW, 78

fill
hdnum::Vector< REAL >, 74

getNeighborIndex
hdnum::SGrid< N, DF, dimension >, 55
getNonlinearProblem
newton.hh, 80
gnuplot
hdnum::Vector< REAL >, 74
gram_schmidt
qr.hh, 86

hdnum::Banach, 7
hdnum::DenseMatrix< REAL >, 11
colsize, 14
identity, 28
mm, 14
mv, 15
operator∗, 16, 17
operator∗=, 18
operator(), 16
operator+, 18
operator+=, 19
operator-, 20
operator-=, 20
operator/=, 21
operator=, 21, 22
readMatrixFromFileDat, 28
readMatrixFromFileMatrixMarket, 29
rowsize, 22
sc, 22
scientific, 23
spd, 30
sr, 23
sub, 24
transpose, 24
umm, 25
umv, 25, 26
update, 27
vandermonde, 30
hdnum::DIRK< M, S >, 31
DIRK, 32, 33
hdnum::EE< M >, 33
hdnum::ErrorException, 34
hdnum::Exception, 35
hdnum::GenericNonlinearProblem< Lambda, Vec >, 35
hdnum::Heun2< M >, 36
hdnum::Heun3< M >, 37
hdnum::IE< M, S >, 38
hdnum::ImplicitRungeKuttaStepProblem< M >, 39
hdnum::InvalidStateException, 40
hdnum::IOError, 41
hdnum::Kutta3< M >, 41
hdnum::MathError, 42
hdnum::ModifiedEuler< M >, 43
hdnum::Newton, 44
hdnum::NotImplemented, 45
hdnum::oc::OpCounter< F >, 46
hdnum::oc::OpCounter< F >::Counters, 10
hdnum::OutOfMemoryError, 47
hdnum::RangeError, 48
hdnum::RE< M, S >, 48
hdnum::RKF45< M >, 50
hdnum::RungeKutta< M, S >, 51
hdnum::RungeKutta4< M >, 53
hdnum::SGrid< N, DF, dimension >, 54
getNeighborIndex, 55
SGrid, 55
hdnum::SparseMatrix< REAL >, 56
begin, 59
cbegin, 59
cend, 59
colsize, 59
end, 60
identity, 60, 64
matchingIdentity, 61
mv, 61
norm_infty, 62
operator∗, 62
operator∗=, 62

operator/=, 62
rowsize, 63
scientific, 63
SparseMatrix, 58
umv, 63
hdnum::SparseMatrix< REAL >::builder, 8
hdnum::SparseMatrix< REAL >::column_index_iterator, 8
hdnum::SparseMatrix< REAL >::const_column_index_iterator, 9
hdnum::SparseMatrix< REAL >::const_row_iterator, 9
hdnum::SparseMatrix< REAL >::row_iterator, 51
hdnum::SquareRootProblem< N >, 65
hdnum::StationarySolver< M >, 66
hdnum::SystemError, 66
hdnum::Timer, 67
hdnum::TimerError, 68
hdnum::Vector< REAL >, 68
fill, 74
gnuplot, 74
operator<<, 74
operator∗, 70
operator+, 71
operator-, 71
operator=, 72
readVectorFromFile, 75
scientific, 72
sub, 73
two_norm, 73
unitvector, 75
HDNUM_ERROR
exceptions.hh, 78
HDNUM_THROW
exceptions.hh, 78

identity
hdnum::DenseMatrix< REAL >, 28
hdnum::SparseMatrix< REAL >, 60, 64

matchingIdentity
hdnum::SparseMatrix< REAL >, 61
mm
hdnum::DenseMatrix< REAL >, 14
modified_gram_schmidt
qr.hh, 87
mv
hdnum::DenseMatrix< REAL >, 15
hdnum::SparseMatrix< REAL >, 61

newton.hh
getNonlinearProblem, 80
norm_infty
hdnum::SparseMatrix< REAL >, 62

operator<<
hdnum::Vector< REAL >, 74
operator∗
hdnum::DenseMatrix< REAL >, 16, 17
hdnum::SparseMatrix< REAL >, 62
hdnum::Vector< REAL >, 70
operator∗=
hdnum::DenseMatrix< REAL >, 18
hdnum::SparseMatrix< REAL >, 62
operator()
hdnum::DenseMatrix< REAL >, 16
operator+
hdnum::DenseMatrix< REAL >, 18
hdnum::Vector< REAL >, 71
operator+=
hdnum::DenseMatrix< REAL >, 19
operator-
hdnum::DenseMatrix< REAL >, 20
hdnum::Vector< REAL >, 71
operator-=
hdnum::DenseMatrix< REAL >, 20
operator/=
hdnum::DenseMatrix< REAL >, 21
hdnum::SparseMatrix< REAL >, 62
operator=
hdnum::DenseMatrix< REAL >, 21, 22
hdnum::Vector< REAL >, 72
ordertest
rungekutta.hh, 93

permute_forward
qr.hh, 88

qr.hh
gram_schmidt, 86
modified_gram_schmidt, 87
permute_forward, 88
qr_gram_schmidt, 88
qr_gram_schmidt_pivoting, 89
qr_gram_schmidt_simple, 90
qr_gram_schmidt
qr.hh, 88
qr_gram_schmidt_pivoting
qr.hh, 89
qr_gram_schmidt_simple
qr.hh, 90
qrhousholder
qrhousholder.hh, 92
qrhousholder.hh
qrhousholder, 92
qrhousholderexplizitQ, 92
qrhousholderexplizitQ
qrhousholder.hh, 92

readMatrixFromFileDat
hdnum::DenseMatrix< REAL >, 28
readMatrixFromFileMatrixMarket
hdnum::DenseMatrix< REAL >, 29
readVectorFromFile
hdnum::Vector< REAL >, 75
rowsize
hdnum::DenseMatrix< REAL >, 22
hdnum::SparseMatrix< REAL >, 63
rungekutta.hh

ordertest, 93

sc
hdnum::DenseMatrix< REAL >, 22
scientific
hdnum::DenseMatrix< REAL >, 23
hdnum::SparseMatrix< REAL >, 63
hdnum::Vector< REAL >, 72
SGrid
hdnum::SGrid< N, DF, dimension >, 55
SparseMatrix
hdnum::SparseMatrix< REAL >, 58
spd
hdnum::DenseMatrix< REAL >, 30
sr
hdnum::DenseMatrix< REAL >, 23
src/exceptions.hh, 77
src/lr.hh, 79
src/newton.hh, 80
src/ode.hh, 81
src/opcounter.hh, 82
src/pde.hh, 85
src/precision.hh, 85
src/qr.hh, 86
src/qrhousholder.hh, 91
src/rungekutta.hh, 93
src/timer.hh, 94
sub
hdnum::DenseMatrix< REAL >, 24
hdnum::Vector< REAL >, 73

transpose
hdnum::DenseMatrix< REAL >, 24
two_norm
hdnum::Vector< REAL >, 73

umm
hdnum::DenseMatrix< REAL >, 25
umv
hdnum::DenseMatrix< REAL >, 25, 26
hdnum::SparseMatrix< REAL >, 63
unitvector
hdnum::Vector< REAL >, 75
update
hdnum::DenseMatrix< REAL >, 27

vandermonde
hdnum::DenseMatrix< REAL >, 30